



UNIVERSITY OF PISA

DEPARTMENT OF INFORMATION ENGINEERING

Master's Degree in Cybersecurity

Design and Secure Deployment of Hierarchical Infrastructures: The ODA Case Study

Supervisors:

Prof. Antonio Brogi
Dott. Alessandro Bocci
Prof. Gian-Luigi Ferrari

Candidate:

Giovanni Neglia

ACADEMIC YEAR 2025/2026

Abstract

The exponential growth of Industry 4.0 has driven a paradigm shift from centralized cloud infrastructures to distributed edge computing. Modern data streaming applications, such as the *Observable Data Access* (ODA) case study, require hierarchical Edge-to-Cloud architectures to scale dependably. However, distributing these microservice-based applications across untrusted transport networks and *Zero Trust* environments introduces critical security vulnerabilities, ranging from typical network eavesdropping to advanced local administrative compromises.

This thesis proposes a comprehensive *Security-as-Code* methodology to expand the standalone ODA ecosystem into a hierarchical architecture. Operating under the constraint that the baseline ODA is treated as a trusted "black box", the deployment of the proposed solution leverages the *Infrastructure as Code* (IaC) paradigm (specifically *Ansi-ble*) not merely as a standard configuration tool, but as a continuous, declarative security orchestrator.

Initially, a macro-architectural Threat Modeling is conducted to evaluate the two distinct architectures proposed: the *Child-Push* and the *Root-Pull* topologies. By profiling specific adversaries and mapping their actions using widely recognized threat modeling frameworks (STRIDE and MITRE ATT&CK), the study identifies the vulnerabilities and the possible mitigations inherent to decentralized versus centralized data replication governance.

Subsequently, these theoretical solutions are translated into an IaC-automated Proof of Concept (PoC). The implemented infrastructures establish a robust cryptographic boundary to secure all inter-node communications, while isolating sensitive deployment secrets at the host level. To effectively contain the blast radius of a potential node takeover, the proposed solutions dynamically enforce fine-grained access policies and resource quotas directly at the application layer. Finally, the architectures ensure continuous resilience through an autonomous self-healing mechanism, designed to detect and revert unauthorized configuration tampering.

To verify the efficacy of these declarative defenses, both architectures are actively subjected to simulated cyber-attacks, and the empirical outcomes confirm the robustness of the applied countermeasures. By contrasting the operational realities of the two replication architectures, the research culminates in a strategic guideline to select the best-suited architectural option according to specific industrial logistics and trust assumptions.

Contents

1	Introduction	1
1.1	Context and Motivation	1
1.2	Objectives	2
1.3	Proposed Solution and Contributions	2
1.4	Thesis Outline	3
2	Background and State of the Art	5
2.1	Observable Data Access (ODA)	5
2.1.1	Interacting with ODA	7
2.1.2	Limitations of the ODA Architecture	8
2.2	Apache Kafka and Kafka MirrorMaker	10
2.2.1	Kafka MirrorMaker 2	10
2.2.2	Kafka Security Mechanics	11
2.3	Ansible and Infrastructure as Code	12
2.4	Threat Modeling Frameworks	13
2.5	Related Work	14
2.5.1	Hierarchical Data Streaming	14
2.5.2	Infrastructure as Code for Security	15
2.5.3	Threat Modeling in Distributed Environments	16
3	Hierarchical ODA Architecture Design	18
3.1	Towards a Hierarchical ODA	18
3.1.1	Security Assumptions and Critical Challenges	19
3.1.2	The Architectural Fork: Push vs. Pull	20
3.2	Child-Push Architecture	20
3.2.1	Operational Advantages	21
3.2.2	Architectural and Security Implications	22
3.3	Root-Pull Architecture	22
3.3.1	Operational Advantages	23

3.3.2	Architectural and Security Implications	24
3.4	Architectural Summary and Security Perspective	25
4	Threat Modeling	27
4.1	System Modeling and Trust Boundaries	28
4.1.1	Data Flow Diagram	28
4.1.2	Trust Boundaries	30
4.2	Threat Identification	31
4.2.1	Attacker Modeling	31
4.2.2	STRIDE and MITRE Mapping	33
4.3	Response and Mitigations	37
4.3.1	Transport Security and Cryptographic Enforcement	37
4.3.2	Secret Management and File System Hardening	38
4.3.3	Replication Governance and Drift Control	38
5	Proof of Concept Implementation	40
5.1	Testbed Environment	40
5.1.1	Virtual Machines Specifications	41
5.1.2	SSH Configuration	41
5.1.3	Certificates Generation	42
5.2	Infrastructure as Code Structure	44
5.2.1	Ansible Repository Architecture	45
5.2.2	Playbook Orchestration	47
5.3	Implementing Security-as-Code Mitigations	49
5.3.1	Transport Security and Cryptographic Enforcement Implementation	49
5.3.2	Secret Management and File System Hardening Implementation	52
5.3.3	Replication Governance and Drift Control Implementation	55
6	Results Validation	61
6.1	Validation Methodology	61
6.2	Threat Mitigation Testing	62
6.2.1	Mitigating the Network Adversary	62
6.2.2	Mitigating the Malicious Observer	65
6.2.3	Mitigating the Compromised Administrator	67
6.3	Replication Performance Benchmarking	75
6.3.1	Child-Push Evaluation	75
6.3.2	Root-Pull Evaluation	75
6.3.3	Benchmarking Summary	76

6.4	Experimental Comparison: Child-Push vs. Root-Pull	76
6.4.1	Network Deployability and the VPN Context	77
6.4.2	Orchestration Complexity and Secret Management	77
6.4.3	Trust Assumptions and Kafka Best Practices	77
6.4.4	Architectural Recommendations	78
7	Conclusions	80
7.1	Summary of Contributions	80
7.2	Architectural Conclusions	81
7.3	Limitations and Future Work	82
A	Appendix	84
A.1	SSH Configuration	84
A.2	Root-Pull Certificates Generation	85

Chapter 1

Introduction

1.1 Context and Motivation

Over the past several years, the exponential growth of the Internet of Things (IoT) and Industry 4.0 has driven a paradigm shift from centralized cloud infrastructures towards distributed edge computing [1, 2]. Modern distributed applications, particularly those handling real-time data streaming, heavily rely on hierarchical architectures to scale dependably across geographically dispersed locations [3, 4]. In these topologies, often structured as a tree from peripheral edge nodes (children) to a central core (root), data must flow seamlessly to ensure low latency, fault tolerance, and bandwidth optimization. However, managing and scaling such a complex, multi-tiered infrastructure introduces significant operational challenges.

To make the provisioning and scaling process reliable and error-proof, organizations have increasingly adopted *Infrastructure as Code* (IaC) methodologies [5, 6]. Tools such as *Ansible* [7] allow administrators to automate the deployment of streaming platforms across tens of nodes, ensuring configuration consistency, idempotency, and rapid recovery from failures.

While IaC solves the operational need for scalability, it inadvertently amplifies security risks if it is not managed correctly [8, 9, 10]. In a hierarchical architecture, edge nodes are often deployed in untrusted physical or network environments, making them highly susceptible to local tampering, configuration drift, and lateral movement attacks. Furthermore, automating a flawed or overly permissive configuration can instantly propagate vulnerabilities across the entire infrastructure.

Therefore, there is a critical need to satisfy the operational requirements of hierarchical data streaming while adopting a *Secure by Design* and *Zero Trust* approach [11, 12]. Security must no longer be an afterthought but must be systematically embedded into the IaC deployment lifecycle. This thesis is motivated by the necessity to bridge the gap be-

tween automated infrastructure scalability and robust security enforcement, demonstrating that hierarchical applications can, and indeed should, be designed without neglecting the fundamental pillars of confidentiality, integrity, and availability.

1.2 Objectives

The primary goal of this thesis is to design, implement, and validate a secure, automated deployment strategy for a hierarchical data streaming infrastructure, the *Observable Data Access* (ODA) [13, 14]. To achieve this, the research is driven by the following specific objectives:

- To define and analyze hierarchical architectural proposals: Specifically, to evaluate the operational and security trade-offs between different replication topologies in the ODA environment, determining their respective advantages and disadvantages.
- To define a *Threat Model* for these new IaC-managed infrastructures: To formalize the trust boundaries of the system, categorize the potential adversaries, and identify the specific threats targeting the transport network and the deployed nodes.
- To provide a Secure by Design *Proof of Concept* (PoC): Translating the theoretical threat model into a practical implementation of the identified architectures, using tailored tools.
- To validate the achieved security outcomes: To conduct a comprehensive security assessment of the PoC through a structured testing methodology, actively simulating the modeled attack vectors to demonstrate the infrastructure's resilience and verify the efficacy of the implemented mitigations.

1.3 Proposed Solution and Contributions

To fulfill the stated objectives, this thesis proposes an automated, declarative deployment methodology to expand the standalone ODA ecosystem into a geographically distributed Edge-to-Cloud architecture. Operating under the constraint that the internal ODA microservices are treated as a trusted "black box", the proposed solutions delegate the security enforcement entirely to the infrastructure and transport layers through a *Security-as-Code* approach.

The core of the proposed solutions relies on *Apache Kafka* [15, 16] and *Kafka MirrorMaker 2* (MM2) [17] as the data replication engine, dynamically orchestrated by Ansible. By leveraging Ansible as a continuous security engine, the deployment pipeline

autonomously provisions the virtualized environments, distributes cryptographic identities, and enforces granular access policies, actively minimizing the risks associated with manual configuration drift in Zero Trust environments.

Specifically, the major contributions of this work are:

- **Hierarchical Architectural Design and Formalization:** The definition and implementation of two distinct hierarchical replication paradigms: the *Child-Push* topology (decentralized governance) and the *Root-Pull* topology (centralized governance). The thesis analyzes their operational trade-offs, specifically regarding network feasibility, core versus edge exposure, and orchestration complexity.
- **Macro-Architectural Threat Modeling:** The formulation of a comprehensive threat model tailored for distributed streaming topologies. By strictly defining three adversarial profiles (the Network Adversary, the Malicious Observer, and the Compromised Administrator) and mapping their actions using the *Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privileges* (STRIDE) [18] and MITRE *Adversarial Tactics, Techniques, and Common Knowledge* (ATT&CK) [19] frameworks, the study exposes the vulnerabilities inherent to each architecture.
- **PoC Security-as-Code Implementation:** The development of a highly modular, idempotent Ansible infrastructure that translates theoretical threat model mitigations into active technical countermeasures. This encompasses internal Public Key Infrastructure (PKI) generation for mutual TLS (mTLS), secret protection via Ansible Vault and strict POSIX file system hardening, dynamic injection of Kafka *Access Control Lists* (ACLs) and *Network Quotas*, and the engineering of event-driven handlers for autonomous configuration drift reconciliation.
- **Results Validation:** The infrastructures' resilience is assessed by actively executing simulated attacks targeting the transport, file system, and concerned application layers. This validation phase culminates in a conclusive architecture selection guideline, achieving the optimal deployment strategy based on specific logistical scenarios and trust assumptions.

1.4 Thesis Outline

To address the complexities of securing hierarchical data streaming environments, the narrative of this manuscript progresses logically from architectural design and theoretical foundations to the practical implementation and validation. The research is organized into the following chapters:

Chapter 2 (Background and State of the Art) establishes the foundational knowledge required for this work. It introduces the baseline ODA architecture, explores the mechanics of Kafka and MM2, defines the Ansible automation engine, and introduces threat modeling frameworks alongside related literature in the field.

Chapter 3 (Hierarchical ODA Architecture Design) analyzes the conceptual transition toward an Edge-to-Cloud continuum. It provides a deep-dive into the operational logistics, network flows, and structural trade-offs between the Child-Push and Root-Pull replication topologies.

Chapter 4 (Threat Modeling) conducts the core security analysis and threat identification. It delineates the system's Trust Boundaries, profiles the potential adversaries, and systematically maps their threats against both architectures, concluding with a strategic baseline of actionable mitigations.

Chapter 5 (Proof of Concept Implementation) transitions the research from theory to practice. It describes the testbed environment, outlines the modular structure of the declarative Ansible repository, and provides a closer technical look at how the Security-as-Code mitigations are programmed and enforced on the Zero Trust nodes.

Chapter 6 (Results Validation) presents the concluding security assessment. It details the execution of targeted cyber-attacks against the provisioned infrastructure to validate mTLS enforcement, credential protection, authorization boundaries, and autonomous drift reconciliation, ultimately providing a comparison and selection strategy between the Child-Push and Root-Pull models.

Chapter 7 (Conclusions) summarizes the key findings of this research, evaluates the achievement of the initial objectives, and outlines potential directions for future work enhancements.

Chapter 2

Background and State of the Art

2.1 Observable Data Access (ODA)

The ODA service [13, 14], whose 1.1 version is depicted in Figure 2.1, is a microservice-based architecture developed within the Network for Energy Sustainable Transition (NEST) Project. It allows Data Generators (DGs) to send data to the service and Data Consumers (DCs) to consume and query the data stored in the service. DGs push data acquired by sensors, digital twins, or services through a streaming broker. ODA provides data access to DCs in two ways: a DC can subscribe to certain topics to get data streamed by generators about such topics; a DC can pull data by invoking a REST API provided by the service.

The service is composed of the following microservices:

- **API Gateway:** the entry point of the service. Provides the Kafka endpoint to Data Generators and Data Consumers and manages the registration of the topics. It also provides the query endpoint to Data Consumers.
- **Apache Kafka:** the message broker, managed by Zookeeper, that allows Data Generators to stream data through ODA and Data Consumers to receive streamed data.
- **Data Pump:** the microservice that, acting as a consumer of the Kafka broker, subscribes to all the Kafka topics, sends the data to be stored to the Database Manager, and retrieves data when requested.
- **Topic Manager:** the microservice that manages the Kafka topics registered in ODA creating them on behalf of DGs and providing the topic list to DCs.
- **Database Manager:** the microservice that manages the InfluxDB database. It stores the data sent by the Data Generators and provides the data to the Data Consumers.

- **InfluxDB**: the time-series database that stores permanently the data sent by the Data Generators.
- **Data Aggregator**: the microservice that manages the aggregation of the data stored in ODA. It allows querying the data stored in ODA and receiving aggregated results.
- **Web Data Transformer**: a web application that allows the creation of mappings of data formats, managing mappings, and performing queries.
- **Data Transformer**: the microservice that manages the mappings.
- **MySQL**: the relational database that stores mappings.

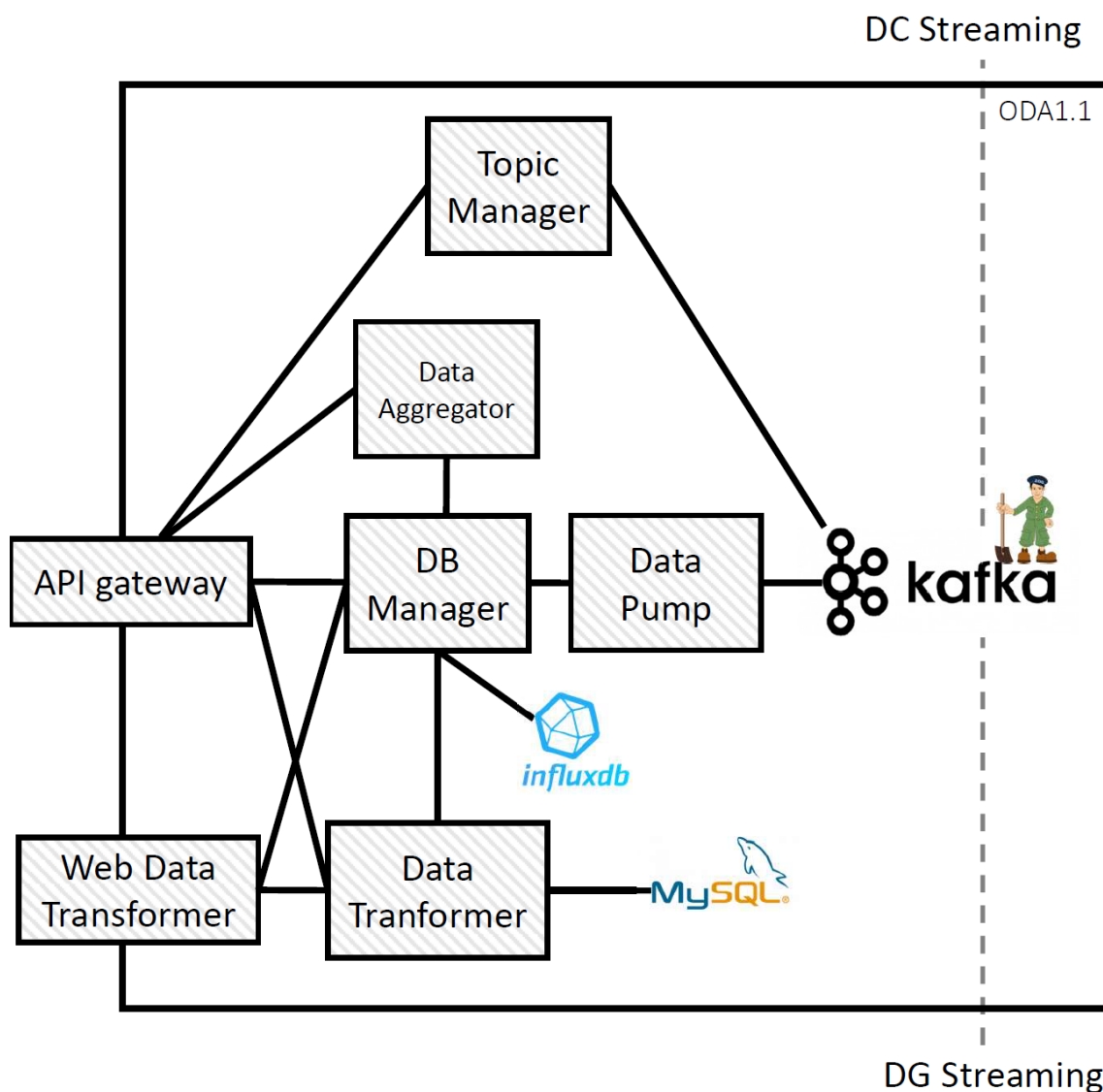


Figure 2.1: ODA 1.1 microservice Architecture.

2.1.1 Interacting with ODA

Both DGs and DCs must register to the ODA service through the API Gateway, which delivers the endpoint to reach the Kafka broker.

Figure 2.2 shows how the DGs act to send data to the ODA service, starting with the registration of the topics they will use and then streaming the data to the Kafka broker directly.

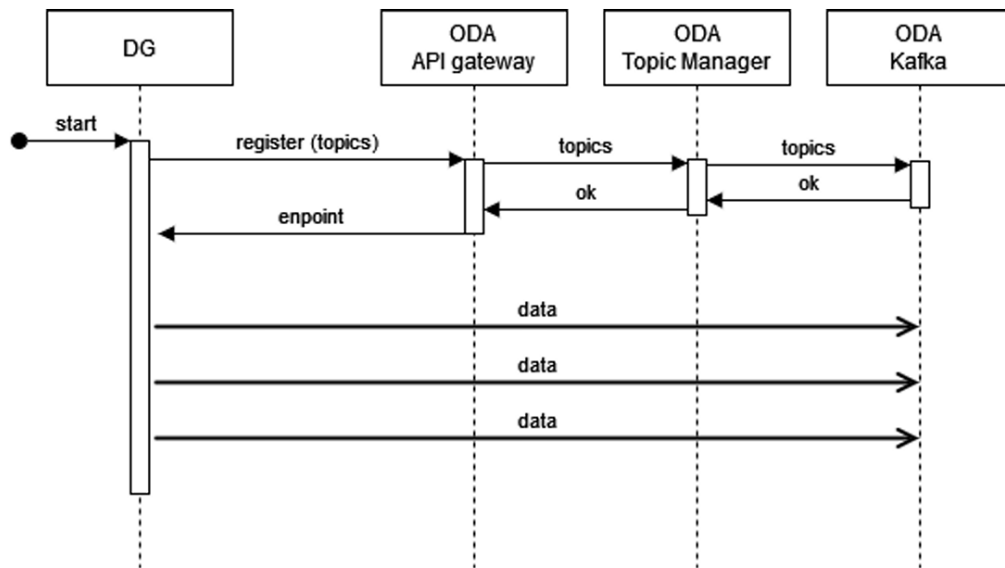


Figure 2.2: A Data Generator streaming data.

Figure 2.3 shows how DCs interested in data streaming must act to receive data from the ODA service: after registration through the API Gateway, DCs receive the list of ODA topics and the Kafka endpoint. They must subscribe to the topics they are interested in by contacting the Kafka broker. From that point on, the data streamed to Kafka on such topics will be sent to DC.

Figure 2.4 shows how the DCs can pull data stored in the InfluxDB. They send a query to the API Gateway, which is propagated to the Database Manager. The data retrieved from the database are sent back to the DC along the reverse path.

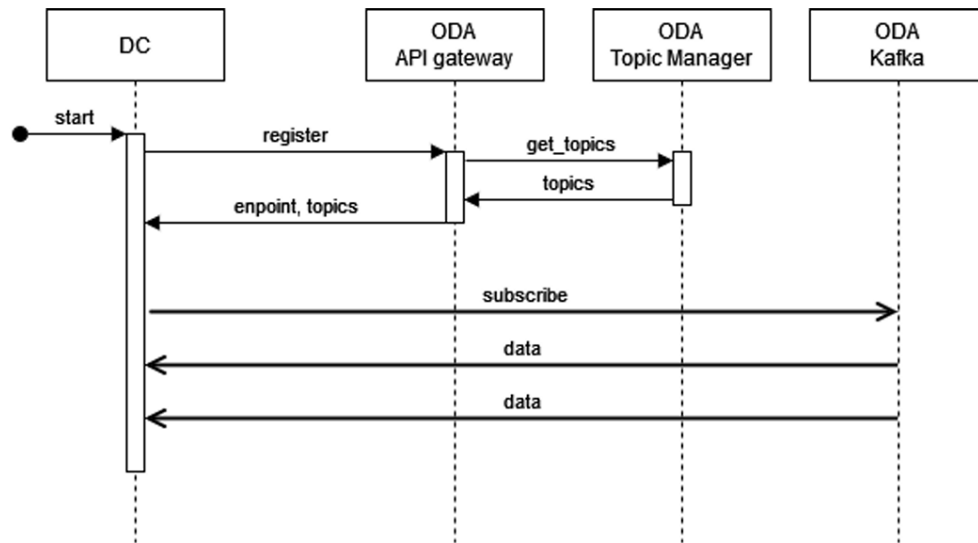


Figure 2.3: A Data Consumer receiving streaming data.

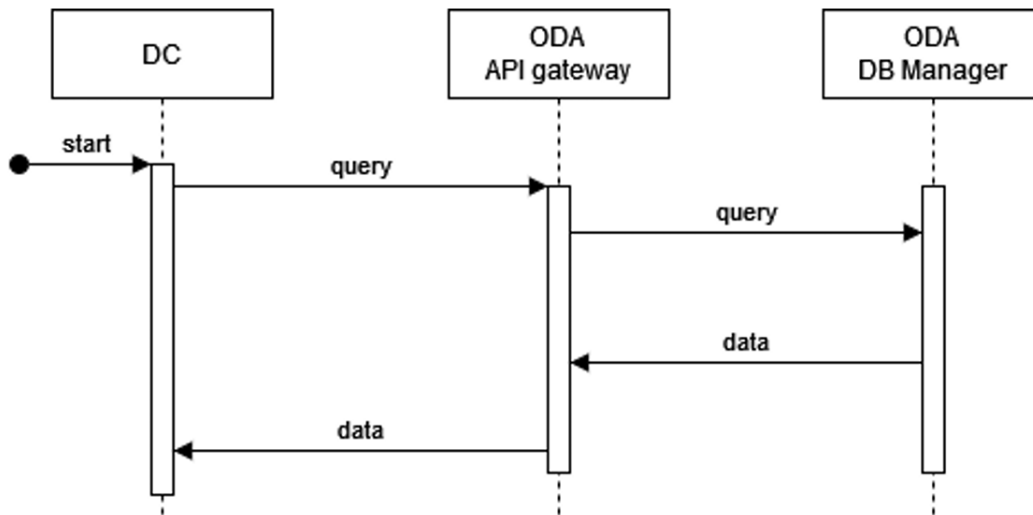


Figure 2.4: A Data Consumer queries data stored in the Database.

2.1.2 Limitations of the ODA Architecture

While the baseline ODA architecture provides a functional microservice ecosystem for local data ingestion and querying, it exhibits significant limitations when considered for enterprise-grade, geographically distributed deployments. These constraints can be categorized into two primary domains: hierarchical scalability and systemic security.

Hierarchical and Distributed Limitations

By design, the baseline ODA service operates as a standalone, self-contained instance managed by a local *Docker Compose* [20] deployment. It completely lacks native protocols for inter-node federation, cross-cluster topic synchronization, or hierarchical data aggregation. In modern distributed scenarios, deploying multiple isolated ODA instances at the Edge without a centralized aggregation strategy inevitably leads to isolated "data silos". To overcome this, and build an Edge-to-Cloud continuum, the architecture requires the integration of a robust replication engine and a centralized orchestration methodology to deploy and manage these interconnected nodes consistently.

Security and Trust Model Limitations

From a security perspective, the baseline ODA architecture operates on a traditional, perimeter-based trust model, implicitly trusting all interactions that occur within its internal Docker network bridge. It was not conceived under a "Secure by Design" methodology. Consequently, the service completely lacks security measures such as native authentication and authorization mechanisms for Data Generators and Data Consumers. Furthermore, there is no strict validation of the generated input data, and inter-service communications occur in plaintext. These flaws make the isolated deployment highly susceptible to local exploitation, data poisoning, and Denial of Service (DoS) attacks, as comprehensively demonstrated in the vulnerability assessment conducted by Venturi [21].

Transitioning this implicitly trusted, standalone architecture into a distributed Edge-to-Cloud ecosystem exponentially amplifies these vulnerabilities. Exposing an unsecured broker directly to a public transport network for inter-node replication would inevitably lead to unauthorized data injection and systemic compromise.

Therefore, establishing a secure hierarchical ODA requires a paradigm shift. Since rewriting the internal source code of the ODA microservices to natively support required security measures falls outside the scope of this work, the baseline application must be encapsulated and treated as a trusted "black box" at the local level. To safely distribute this system, the security enforcement is instead elevated to the infrastructure and transport layers, ensuring that the distributed replication pipeline is secured without altering the underlying microservice logic.

2.2 Apache Kafka and Kafka MirrorMaker

To guarantee a high-throughput and low-latency data streaming, the ODA service leverages Apache Kafka¹ as its core message broker. Kafka is an open-source, distributed event streaming platform designed to handle real time data feeds efficiently [15, 16].

Inside the ODA ecosystem, Kafka acts as the central nervous system for data ingestion and routing. Its operation relies on two fundamental concepts:

- **Broker:** is the server instance responsible for receiving, storing, and forwarding messages. In the ODA architecture, a single Kafka broker works in the isolated environment of each node along the microservices mentioned above.
- **Topic:** is a logical "folder" in which events are published. Topics in Kafka are always multi-producer and multi-subscriber giving the possibility to read data as often as needed. In ODA, topics can represent specific data streams (e.g., a sensor telemetry) and the single broker approach simplifies the overall application architecture.

Although Kafka is intrinsically built to scale horizontally across large clusters using concepts such as partitions and replicas, the specific deployment within the ODA context is reduced to a single broker architecture as it does not require these features. This architectural choice minimizes resource overhead while still providing the robust publish-subscribe mechanics required by the ODA service.

2.2.1 Kafka MirrorMaker 2

While a single Kafka broker handles data flow within a local ODA node, a hierarchical multi-node architecture requires data to be possibly synchronized between geographically separated environments.

To bridge this gap, the Kafka ecosystem provides Kafka MirrorMaker 2 [17]. MM2 is a utility built upon the Kafka Connect framework, specifically designed to replicate data between different Kafka nodes in a streaming manner [22]. Kafka administrators can replicate topics and relevant metadata from one or more source Kafka clusters to one or more target Kafka clusters. In summary, MM2 uses connectors to consume from source clusters and produce to target clusters as displayed in Figure 2.5.

Because it natively supports aggregation and complex topologies, MM2 is the designated solution to aggregate data from edge nodes into a central unit, making it particularly relevant for hierarchical architectures. In the context of this thesis, MM2 is the designated engine that enables data replication.

¹In this work, a custom Kafka Docker image was provided, bundled with Kafka version 3.6.1.

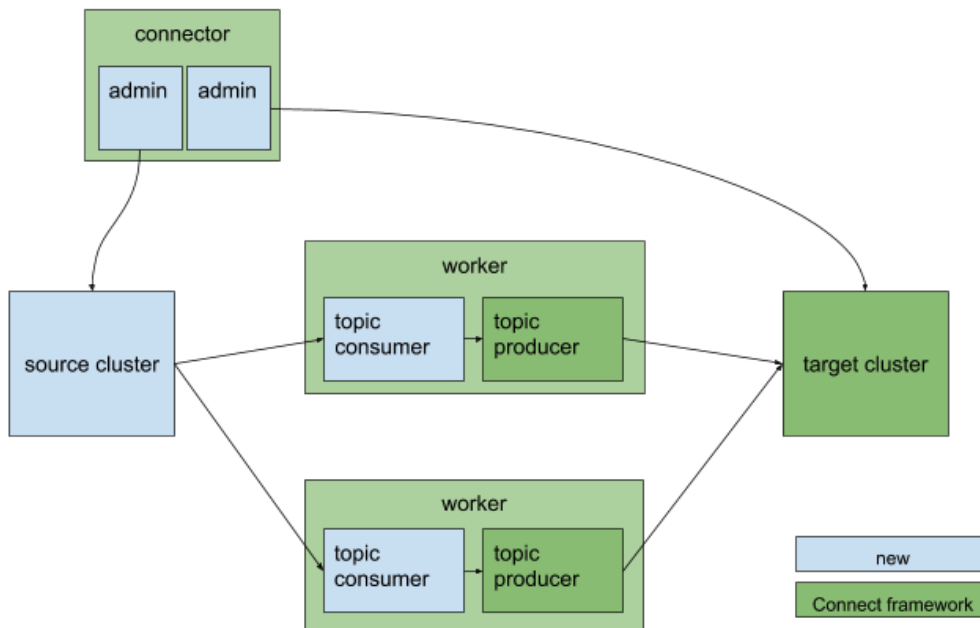


Figure 2.5: MirrorSourceConnector and its workers, contains a pair of producers and consumers to replicate records.

2.2.2 Kafka Security Mechanics

Deploying Kafka across untrusted, distributed networks requires strict security enforcement beyond basic data replication. Kafka natively provides several layers of protection that are fundamental to securing external boundaries:

- **Mutual TLS (mTLS):** Kafka supports the Transport Layer Security protocol not only to encrypt data in transit but also to enforce mutual client-server authentication via cryptographic certificates, effectively neutralizing network eavesdropping and spoofing.
- **Access Control Lists (ACLs):** Kafka features an authorizer capable of managing fine-grained permissions. ACLs dictate exactly which authenticated principal is allowed to read from or write to specific topics, enforcing the principle of least privilege.
- **Network Quotas:** To prevent resource exhaustion, Kafka allows administrators to enforce bandwidth quotas (e.g., maximum byte rate per second) on specific clients or groups, mitigating the risk of application-level volumetric DoS attacks (e.g., malicious data flooding).

2.3 Ansible and Infrastructure as Code

As distributed systems face the challenge to scale across multiple nodes, manual configuration becomes highly error-prone and difficult to secure. To address this, modern operational environments adopt Infrastructure as Code methodologies, which involve managing and provisioning computing infrastructure through machine-readable definition files rather than ad-hoc source code changes or interactive configuration tools.

In our scenario, Ansible [7] is employed as the IaC tool to automate the deployment and secure configuration of the hierarchical ODA structure. Developed by Red Hat, Ansible is an open source automation engine that facilitates cloud provisioning, configuration management, and application deployment. Unlike other configuration management tools that require custom agents to be installed on managed nodes, Ansible utilizes an agentless architecture, connecting to the target machines using standard Secure Shell (SSH) and pushing small programs, called *modules*, to them. These modules are executed on the nodes and then removed when finished.

The operation of Ansible revolves around several key concepts:

- **Playbooks:** written in YAML, playbooks are the core configuration, deployment, and orchestration language in Ansible. They describe the desired state of the target systems in a human-readable format.
- **Inventory:** is a list of managed hosts and groups of hosts upon which commands, modules, and tasks in a playbook operate.
- **Roles:** provide a framework for fully independent, or interdependent, collections of variables, tasks, files, templates, and modules. They allow for the reuse of configuration logic and the logical grouping of infrastructure components, keeping playbooks clean and modular.
- **Tasks and Modules:** a task is a single unit of action (e.g., copying a file, starting a Docker container, setting file permissions) executed by a specific module.
- **Idempotency:** Ansible operations are designed to be idempotent, meaning executing a playbook multiple times will not change the system beyond the desired initial state.
- **Ansible Vault:** is a feature that allows users to keep sensitive data such as passwords, private keys, and SSL certificates in encrypted files, rather than as plaintext in playbooks or roles.

2.4 Threat Modeling Frameworks

Once the hierarchical architectures have been designed, we will address the question of "what can go wrong" focusing on identifying and ranking cyberthreats within the context of our systems. In an attempt to answer this question, the STRIDE framework will be used.

STRIDE [18], originally developed by Microsoft employees, is a mature and popular threat modeling technique and continues to be a reference point in modern processes [23, 24] due to its highly flexible approach. To facilitate threat identification, STRIDE groups threats into one of six general prompts and engineers are encouraged to systematically consider how these general threats may materialize within the context of the specific system being evaluated. STRIDE categories and associated desirable attributes are described in Table 2.1.

Threat Category	Violates	Examples
Spoofing	Authentication	An attacker steals the authentication token of a legitimate user and uses it to impersonate the user.
Tampering	Integrity	An attacker abuses the application to perform unintended updates to a database.
Repudiation	Accounting	An attacker manipulates logs to cover their actions.
Information Disclosure	Confidentiality	An attacker extracts data from a database containing user account info.
Denial of Service	Availability	An attacker locks a legitimate user out of their account by performing many failed authentication attempts.
Elevation of Privileges	Authorization	An attacker tampers with a JWT to change their role.

Table 2.1: STRIDE threat model categories and examples.

To provide a more actionable and standardized classification of how these theoretical STRIDE threats are technically executed, this work will complement the analysis by mapping the identified risks to the MITRE ATT&CK framework [19]. This integration allows for a precise description of the exact operational techniques (e.g., Network Sniffing, Credential Harvesting) an adversary might employ.

2.5 Related Work

2.5.1 Hierarchical Data Streaming

The necessity of seamlessly connecting peripheral edge devices to centralized cloud infrastructure has driven significant research into distributed data streaming platforms. A highly relevant contribution in this domain is the Octopus framework proposed by Pan et al. [4], which implements a hybrid, cloud-to-edge event-driven architecture designed for distributed scientific computing. To handle high-throughput and reliable asynchronous communication, Octopus leverages Apache Kafka as its core messaging fabric and can employ Kafka MirrorMaker to replicate and synchronize topics across distributed environments.

The Octopus framework successfully demonstrates the operational viability of aggregating data from geographically dispersed edge nodes into a central cloud broker, which closely aligns with the data flow characteristics of the Child-Push topology analyzed in this thesis. However, their implementation heavily relies on fully managed cloud services, such as Amazon Web Services (AWS) Managed Streaming for Kafka and proprietary Identity and Access Management policies, effectively abstracting away the underlying infrastructure and its related operational risks. While this approach guarantees high performance and simplifies access control, it does not address the fundamental security challenges of deploying and managing untrusted, Zero Trust edge nodes in self-hosted environments.

In the specific context of Industry 4.0, the architectural challenge of synchronizing edge deployments with central systems is often framed as the integration between Operational Technology (OT) and Information Technology (IT) domains. Along this line, Bosi et al. [3] proposed a layered data integration platform compliant with the Reference Architectural Model Industrie 4.0 (RAMI 4.0). Similar to the Octopus framework, their architecture relies on Kafka to decouple data producers from consumers. Notably, to safely bridge the segregated domains, the authors explicitly employ MirrorMaker to establish a one-way data mirroring pipeline from the peripheral OT layer to the centralized IT layer, ensuring that industrial machines remain isolated from direct external IT interactions.

However, its security paradigm is entirely network centric. The authors operate under the strict assumption that the peripheral OT layer is deployed in a highly secure, trusted network domain, physically and logically protected from tampering. For deployments, they rely on standard container orchestration tools, such as Kubernetes, primarily for ease of maintenance rather than security enforcement. By stripping away the assumption of an inherently secure edge, this thesis expands upon such integration models. We introduce Ansible not merely as a deployment script, but as a continuous Security-as-Code engine to dynamically harden the Kafka brokers, manage access control lists, and enforce the

desired security state directly at the Zero Trust edge.

2.5.2 Infrastructure as Code for Security

The paradigm shift from manual infrastructure configuration to IaC has introduced new methodologies for managing complex, distributed systems. In their systematic review of IaC technologies, Pahl et al. [5] highlight that one of the most critical operational challenges in modern deployments is configuration drift, the phenomenon where the actual state of a system diverges from its intended baseline due to unauthorized or ad-hoc manual modifications. The authors note that unidentified configuration drift poses severe risks, including massive outages and security vulnerabilities.

Furthermore, the review provides a structural taxonomy of IaC tools, emphasizing the architectural distinction between agent-based systems (e.g., Puppet, Chef) and agentless systems like Ansible, which orchestrates remote nodes directly over SSH without requiring dedicated listening daemons on the target machines.

Although this study formalizes these concepts from a general lifecycle and *Development, Security, and Operations* (DevSecOps) perspective, this thesis actively leverages these architectural traits as explicit defensive countermeasures against the threat profiles typical of Edge-to-Cloud data streaming.

Beyond infrastructural hardening and active drift mitigation, the static IaC codebase itself introduces novel vulnerabilities if not properly managed. Rahman et al. [8] highlighted this critical issue in their empirical study on IaC, pointing out that deployment scripts are highly susceptible to containing secrets, due to the prevalent anti-pattern of hard-coding passwords and cryptographic keys. To mitigate this, the authors derived a set of secret management practices, emphasizing the necessity of prioritized encryption, dedicated password management for tools like Ansible Vault, and strict access control policies to protect secrets within automated provisioning pipelines.

To adhere to the best practices identified by Rahman et al., this work integrates Ansible Vault directly into the automated deployment workflow. By encrypting sensitive variables, the proposed Security-as-Code architecture ensures that cryptographic materials are not exposed as plaintext within the centralized IaC repository. During the execution phase, these secrets are decrypted on-the-fly into the target node's volatile memory. While the configuration files containing these decrypted secrets must ultimately be materialized on the storage of the untrusted nodes to allow application startup, they are protected leveraging the aforementioned access control practice.

2.5.3 Threat Modeling in Distributed Environments

The architectural shift towards decentralized and microservice-based ecosystems has significantly complicated traditional threat modeling paradigms. Modern distributed components and APIs require hybrid modeling workflows to effectively trace vulnerabilities and dependency across complex, interconnected boundaries. As highlighted by Bezerra et al. [25], standard methodologies are often insufficient for these complex systems, prompting the development of hybrid approaches, such as the HyVAW methodology, which aggregate frameworks like STRIDE, Common Vulnerability Scoring System (CVSS), and Attack Defense Trees to systematically model distributed threats.

While such methodologies provide a comprehensive theoretical workflow for mapping vulnerabilities in distributed microservices, they focus heavily on software-level visualization and risk calculation. Consequently, they often leave a critical "implementation gap" regarding the dynamic, automated enforcement of countermeasures at the infrastructure level.

This specific challenge is formally recognized within the DevSecOps paradigm as the "Automation Enigma". In their extensive review of threat models adapted for continuous delivery pipelines, Nagasundari et al. [26] highlighted the difficulty of automating threat identification and mitigation across heterogeneous, distributed environments due to the lack of standardized enforcement tools. Furthermore, while their study proposes a layered security architecture to address this enigma in multi-cloud environments, it heavily relies on abstracted, vendor-provided managed services, effectively leaving physically dispersed Edge nodes unaddressed.

A foundational step towards securing the ODA ecosystem was recently taken by Venturi [21], who conducted a comprehensive vulnerability assessment and threat modeling of a standalone ODA instance. Using the STRIDE framework, the author evaluated the microservice-based architecture against both external and internal adversaries. The study highlighted critical intra-node vulnerabilities, such as the lack of internal authentication, container misconfiguration, and susceptibility to local DoS attacks within the isolated Docker deployment.

While Venturi's work provides a rigorous micro-level security analysis of the base ODA components, it is strictly confined to a single, isolated node. However, modern Industry 4.0 and IoT deployments necessitate distributed, hierarchical topologies spanning from the Edge to the Cloud. This architectural evolution introduces a fundamentally different attack surface. In such distributed environments, the primary security concerns shift from intra-service communication to inter-node data replication over untrusted transport networks and the management of geographically dispersed, Zero Trust edge nodes.

This thesis bridges this research gap by elevating the threat model from the microservice

level to the macro-architectural level. Building upon the baseline assumption that the internal ODA microservices are encapsulated entities treated as trusted "black boxes", this work addresses the novel risks introduced by both the Child-Push and Root-Pull replication topologies. Furthermore, unlike previous approaches that focus purely on vulnerability discovery and testing, this thesis proposes a proactive, automated mitigation strategy.

Chapter 3

Hierarchical ODA Architecture Design

3.1 Towards a Hierarchical ODA

The baseline ODA was designed to operate as a self-contained unit, managed directly through a Docker Compose file, capable of handling data ingestion and storage within a localized environment. However, modern IoT and Industry 4.0 deployments rarely operate in isolation. They typically require a distributed hierarchy where multiple peripheral edge nodes (the *Children*) collect and preprocess sensor data before forwarding it to a centralized node (the *Root*) for long-term storage, global aggregation, and advanced analytics.

To visualize this paradigm shift, Figure 3.1 illustrates the conceptual Edge-to-Cloud continuum of the Hierarchical ODA ecosystem.

The focus of this thesis is the design and security validation of this new hierarchical extension. Transitioning to this distributed model provides several fundamental operational advantages:

- **Bandwidth Optimization:** Transmitting raw, high-frequency data directly from thousands of IoT sensors to a central cloud is often unfeasible. Edge nodes act as local buffers, allowing for data filtering or aggregation before traversing the Wide Area Network (WAN).
- **Low Latency and Fault Tolerance:** Local Data Consumers can interact with the peripheral Child nodes in real-time without relying on external internet connectivity. If the connection to the core Root node drops, the Edge can continue to ingest and buffer data locally, ensuring no data is lost during outages.
- **Centralized Analytics:** The Root node provides a unified, global view of the entire distributed fleet, enabling complex analytics and long-term historical storage that would exceed the physical capacities of the peripheral hardware.

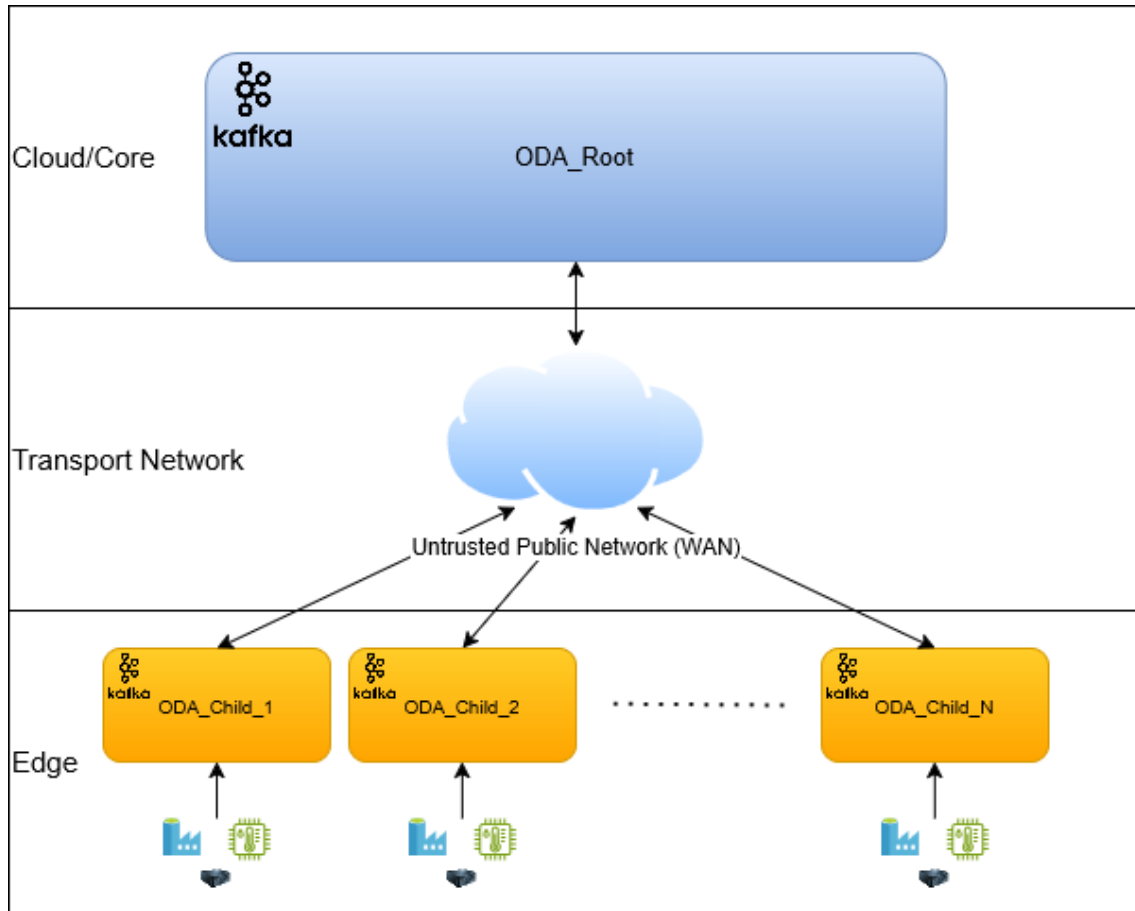


Figure 3.1: Conceptual representation of the Hierarchical ODA Edge-to-Cloud ecosystem.

3.1.1 Security Assumptions and Critical Challenges

While the operational benefits are substantial, deploying a hierarchical architecture significantly alters the attack surface. To properly design the replication pipelines and lay the groundwork for the Threat Model detailed in Chapter 4, several security assumptions must be established:

- **Untrusted Transport Network:** The data flowing between the Child nodes and the Root node traverses public networks or shared Virtual Private Networks (VPNs). It must be assumed that the communication channel is intrinsically insecure and susceptible to interception and manipulation.
- **Zero Trust Environments:** Unlike the central Root, which is typically hosted in a secure Cloud or on-premise datacenter, Child nodes are deployed in peripheral, physically accessible locations (e.g., factory floors, remote sites). They must be treated as "Zero Trust" environments, highly vulnerable to local tampering and credential harvesting.

- **Trusted Baseline Microservices:** As defined in Section 2.1.2, the internal ODA microservices (API Gateway, InfluxDB, etc.) are treated as encapsulated "black boxes." The security design must protect the ecosystem at the infrastructure and transport layers without relying on internal application-level modifications.

3.1.2 The Architectural Fork: Push vs. Pull

To evolve ODA into this multi-tier system while respecting the aforementioned constraints, a robust replication mechanism is required. As introduced in Chapter 2, MM2 serves as the engine for this synchronization. Furthermore, manually deploying and managing multiple edge nodes is operationally infeasible and highly prone to configuration drift. Therefore, the architectural design mandates the use of an IaC orchestrator, specifically Ansible, to provision the nodes, configure the Kafka brokers, and establish the replication flow consistently.

Depending strictly on where the MM2 engine is deployed, the deployment strategy forks into two distinct architectural topologies: the Child-Push architecture and the Root-Pull architecture.

Both approaches leverage the exact same underlying technologies but present fundamentally different network flows, performance profiles, and security trade-offs. It is important to clarify that this naming convention is purely semantic and adopted throughout this thesis to illustrate the data flow from an architectural standpoint. At a low level, the MM2 engine invariably operates by consuming (pulling) data from a source broker and producing (pushing) it to a target broker, regardless of where the process is physically deployed. The terms "Push" and "Pull" are used here strictly to denote which node hosts the replication engine. Sections 3.2 and 3.3 will analyze these two competing paradigms in detail.

3.2 Child-Push Architecture

In the Child-Push architecture, depicted in Figure 3.2, the replication engine and its associated governance are decentralized. The MM2 process is integrated directly into the deployment configuration (the Docker Compose stack) of each peripheral Child node, alongside the local ODA microservices.

In this configuration, the Child node assumes the role of the active network client, initiating the data transfer. The workflow operates as follows:

1. The MM2 process running on the Child continuously polls and consumes data from its local Kafka broker.

2. MM2 initiates an outbound network connection targeting the publicly exposed Kafka endpoint on the central Root node.
3. The Child actively "pushes" (produces) the selected topic data into the Root's Kafka broker. To prevent data collision among multiple children synchronizing simultaneously, the pushed topics are typically prefixed with the specific Child's identifier (e.g., `oda_child_1.testingtopic`).

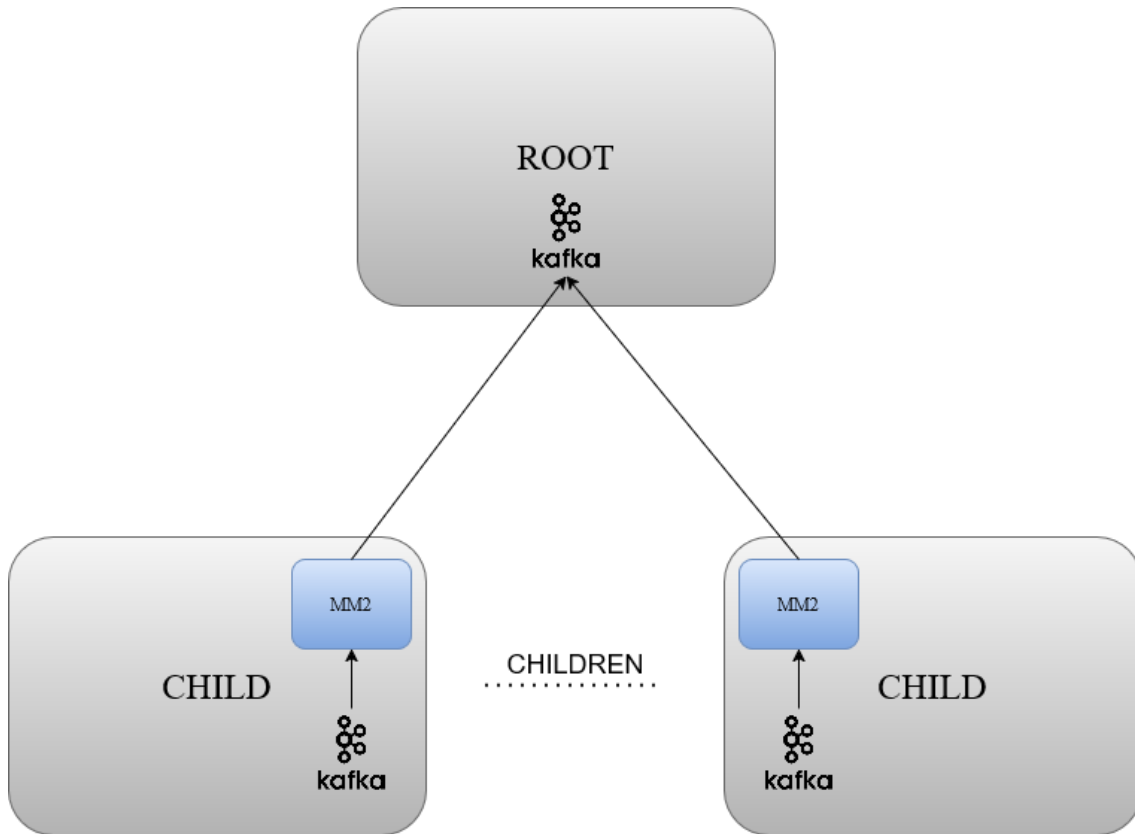


Figure 3.2: Child-Push Architecture.

3.2.1 Operational Advantages

From a networking and deployment perspective, the Child-Push model offers distinct operational advantages for Edge computing environments. Peripheral nodes are frequently deployed in challenging infrastructure setups, such as behind corporate firewalls, Network Address Translation (NAT) gateways, or on mobile 4G/5G networks where they are assigned dynamic, private IP addresses.

Because the Child initiates the connection (egress traffic), the communication naturally traverses NAT and stateful firewalls without requiring complex port-forwarding rules or dedicated VPNs at the edge. Consequently, the central Root node only needs a single,

stable public endpoint to asynchronously receive data from a number of Edge nodes. This makes the architecture highly scalable and relatively simple to provision from a networking standpoint.

3.2.2 Architectural and Security Implications

While solving edge reachability issues, this topology introduces critical security challenges and shifts the burden of defense heavily onto the central infrastructure:

- **Root Exposure and Core Vulnerability:** To allow inbound replication, the central Root node must expose its Kafka broker directly to the transport network. This wide-open ingress traffic exposes the core infrastructure to external scanning, unauthorized application-level access attempts, and resource exhaustion.
- **Decentralized Replication Governance:** Because the MM2 engine runs on the Child, the configuration defining *what* to replicate (topic whitelists) is managed locally at the edge. If a Child node is compromised, the attacker gains total control over this pipeline. Without strict, centralized countermeasures, a rogue edge node could flood the Root with garbage data, exhaust the core's storage, or maliciously alter the topics being pushed.
- **Producer Network Sensitivity:** According to official Kafka documentation [17], this setup violates the "consume from remote, produce to local" best practice. Kafka producers inherently struggle more with unreliable, high-latency, or dropping network connections than consumers do, potentially leading to increased message buffering and memory exhaustion on the resource-constrained Edge nodes during network outages.

To make this architecture viable, the core Root node must implement a rigid Zero Trust posture, dynamically enforcing strict identity verification, granular ACLs, and bandwidth quotas on every incoming client connection.

3.3 Root-Pull Architecture

In the Root-Pull architecture, illustrated in Figure 3.3, the replication engine and its governance are completely inverted and centralized. The Edge nodes are stripped of their replication responsibilities. Instead, a distinct, dedicated MM2 process is instantiated on the Root node for every connected Child node.

In this paradigm, the central Root node acts as the active network client, while the geographically dispersed Child brokers act as passive listening servers:

1. The centrally located MM2 processes initiate outbound network connections targeting the exposed public endpoints of their respective remote Child nodes.
2. The Root actively "pulls" (consumes) the data from the Edge brokers across the network.
3. The various MM2 instances locally write the collected data directly into the internal Root broker, managing topic prefixes internally to maintain data segregation.

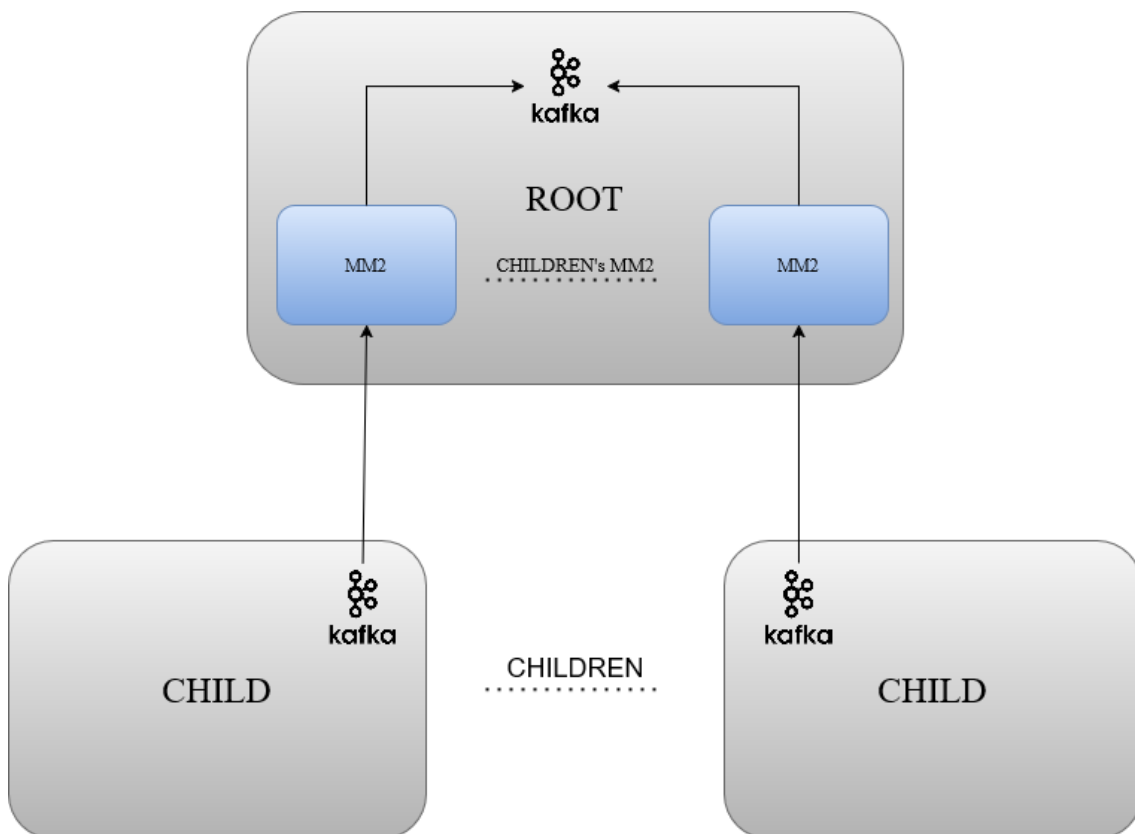


Figure 3.3: Root-Pull Architecture.

3.3.1 Operational Advantages

The primary advantage of the Root-Pull architecture lies in its *Centralized Governance* and the intrinsic protection it provides to the core infrastructure.

First, the Root node does not need to expose its Kafka broker to the outside world for the replication process. Since the Root only makes outbound connections, its core storage and ingestion components remain hidden behind the central firewall, effectively neutralizing inbound external network threats.

Second, the replication logic is managed entirely at the center. The Root explicitly dictates which topics are pulled, when they are pulled, and at what rate. If an Edge node is physically compromised or hijacked, the attacker cannot force arbitrary data into the Root's infrastructure or alter the replication flow. The central node can instantly "quarantine" the breach simply by halting the specific pull process, significantly reducing the systemic blast radius of an edge compromise.

Furthermore, this architecture aligns perfectly with the Kafka best practice to "consume from remote, produce to local". By locating the MM2 processes on the Root node (the target cluster) and consuming data from the remote Children, producer latency is minimized, and the resources of the central cloud can effectively manage any network buffering or retries.

3.3.2 Architectural and Security Implications

Despite the enhanced protection of the core and superior pipeline governance, the Root-Pull topology suffers from severe logistical constraints and introduces a highly complex scaling scenario:

- **Network Infeasibility at the Edge:** The Root must be able to actively route traffic to the IP address of every single Child. This is incompatible with Edge nodes relying on dynamic IPs or standard NAT gateways. Establishing this topology almost always requires the implementation of a complex VPN overlay to make the Edge nodes addressable from the center.
- **Edge Exposure:** While the Root is hidden, every single Child node must now expose its Kafka broker to the transport network to allow the Root to connect. This distributes the attack surface across multiple, physically untrusted locations, requiring every peripheral node to maintain a hardened listening posture.
- **1-to-N Orchestration Complexity:** Centralizing the MM2 engines means the Root node must securely host and execute N distinct replication processes. To preserve the principle of Least Privilege, the Root cannot use a single "master key" to access all children; it requires a unique, granular cryptographic identity to authenticate against each specific Edge server. Orchestrating this dynamic generation of containers and granular secrets without causing configuration drift represents a significant engineering challenge.

3.4 Architectural Summary and Security Perspective

The evaluation of the Child-Push and Root-Pull architectures reveals a fundamental trade-off between operational network feasibility and centralized security control. To provide a clear, direct overview of these competing paradigms, Table 3.1 summarizes their core characteristics across key infrastructural and security dimensions.

Dimension	Child-Push Architecture	Root-Pull Architecture
Network Feasibility	High (Natively traverses Edge NAT/Firewalls via egress traffic).	Low (Requires VPN overlay or static IPs to reach Edge nodes).
Core Exposure	Exposed (Root must accept inbound connections from untrusted networks).	Hidden (Root initiates only outbound connections).
Edge Exposure	Hidden (Children initiate only outbound connections).	Exposed (Children must accept inbound connections).
Replication Governance	Decentralized at the Edge (High risk of rogue node data flooding).	Centralized at the Core (Easy to quarantine compromised nodes).
Orchestration Complexity	Low-Medium (Uniform MM2 deployments at the Edge).	High (1-to-N dynamic MM2 container scaling and granular secrets on Root).
Kafka Best Practices	Violates "Consume remote, produce local" (Higher producer latency).	Adheres to best practices (Robust local producing).

Table 3.1: Direct comparison of Child-Push and Root-Pull operational and security trade-offs.

Because neither architecture provides a definitive "silver bullet" solution, relying solely on theoretical operational advantages is insufficient for a Secure by Design deployment. Consequently, both topologies have been selected for practical implementation within the scope of this thesis.

By deploying both the Child-Push and Root-Pull models, we establish a foundation

for a comprehensive experimental comparison. This dual-implementation strategy allows us to practically evaluate the logistical orchestration constraints, directly compare their resilience against network anomalies, and validate the efficacy of the IaC security countermeasures applied to each scenario.

However, to fully comprehend the security implications of these implementations and design the appropriate automated defenses, it is first necessary to identify, categorize, and evaluate the specific cyber-threats that target their respective network flows and components. Therefore, Chapter 4 will conduct a formal Threat Modeling analysis of both configurations, setting the stage for the practical implementations.

Chapter 4

Threat Modeling

As *Open Worldwide Application Security Project* (OWASP) asserts [24], Threat Modeling is a structured, repeatable process used to gain actionable insights into the security characteristics of a particular system. It involves modeling a system from a security perspective, identifying applicable threats based on this model, and determining responses to these threats. To achieve this, Threat Modeling analyzes a system from an adversarial perspective, focusing on ways in which an attacker can exploit a system.

Threat Modeling is ideally performed early in the *Software Development Life Cycle* (SDLC), as in our case during the design phase of our ODA expansion. This proactive approach does not merely align with general industry best practices, but is expressly mandated by the National Institute of Standards and Technology (NIST) in its *Secure Software Development Framework* (SSDF) [11]. Indeed, the SSDF explicitly cites Threat Modeling as a fundamental task for designing secure software, emphasizing the critical necessity of integrating such practices continuously throughout the SDLC to minimize the number and impact of vulnerabilities. By adhering to these core principles, security is "built-into" a system rather than "bolted-on" as an afterthought, elevating Threat Modeling from an optional add-on to a fundamental engineering requirement.

Specifically, because the Hierarchical ODA ecosystem is fundamentally a distributed data streaming pipeline, this work adopts a *Data-Centric* approach to threat modeling. As outlined by the NIST in *Guide to Data-Centric System Threat Modeling* [27], conventional security methodologies, which primarily focus on preventing generic host or application compromise, are often insufficient for safeguarding high value data within complex topologies. Instead, a data-centric methodology prioritizes the identification of threats targeting the data itself throughout its lifecycle, particularly as it moves across untrusted boundaries.

There is no universally accepted industry standard for the threat modeling process, no "right" answer for every use case. However, despite this diversity, most approaches include the processes of system modeling, threat identification, and risk response in some

form.

As stated in the Threat Modeling Manifesto [28], the following four questions provide the framework that guides the analysis throughout this work:

1. **"What are we working on?":** Building upon the architectural analysis evaluated in Chapter 3, Section 4.1 will formalize the system model for both the Child-Push and Root-Pull topologies. By utilizing a *Data Flow Diagram* (DFD), we will clearly delineate the interacting components and explicitly define the *Trust Boundaries*.
2. **"What can go wrong?":** Following the system modeling, Section 4.2 will focus on *Threat Identification*. We will define the profiles of our potential attackers and then apply the STRIDE framework, integrated with MITRE's ATT&CK techniques, to map the specific threats targeting both architectures.
3. **"What are we going to do about it?":** Once the adversarial threats for both topologies are identified, we will propose a comprehensive set of actionable security mitigations. These countermeasures will form the strategic baseline required to secure the chosen deployment, seamlessly bridging the theoretical analysis with the practical PoC implementation in Chapter 5.
4. **"Did we do a good job?":** The final assessment of our threat model and the effectiveness of the applied countermeasures will be addressed in Chapter 6. Prior to the conclusions, we will conduct a validation of the PoC to ensure that the identified risks have been successfully mitigated in the real world deployment.

4.1 System Modeling and Trust Boundaries

Threat Modeling requires a deep understanding of the system being evaluated. To properly perform this analysis, one must understand data flows, trust boundaries, and the structural characteristics of the environment. Consequently, establishing improved visibility into the system and its interactions is a critical preliminary advantage of this phase.

4.1.1 Data Flow Diagram

As highlighted by Adam Shostack [29], a leading authority in the field of Threat Modeling, DFDs are widely recognized as the standard and most effective approach to visually model a system for security purposes. DFDs allow engineers to visually model a system and its interactions with data and external entities. They facilitate the identification of vulnerable components by highlighting critical intersections, such as data entering or

leaving the system, storage mechanisms, and the flow of control across these boundaries. Due to their hierarchical nature, DFDs can be used to decompose complex applications into manageable lower-level subsystems. For the purpose of this analysis, a high-level context DFD is utilized to clarify the macro-architectural scope of the distributed ODA environment.

Depicted in Figure 4.1, the high-level DFD illustrates the foundational entities and interactions that are common to both the Child-Push and Root-Pull architectures.

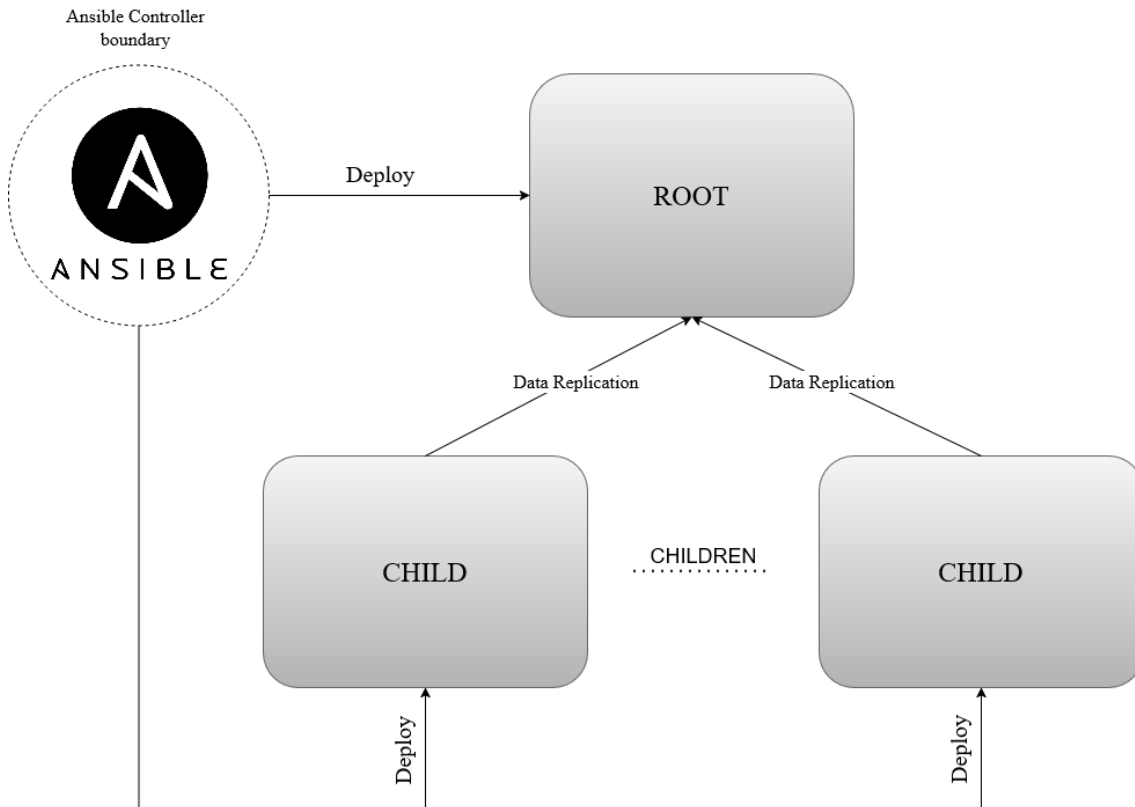


Figure 4.1: Hierarchical ODA Data Flow Diagram.

The system model is built upon three primary logical entities. At the core of the management infrastructure lies the *Ansible Controller*, a centralized workstation responsible for provisioning and configuring the ODA environment via IaC. The target deployments governed by this controller are structurally divided into a centralized core and a distributed periphery. Specifically, the *Root Node* acts as the central Cloud instance, engineered to aggregate and store the synchronized data. Conversely, the *Child Nodes* serve as the peripheral Edge instances, deployed for localized data ingestion and acting as the source of the forwarding pipeline.

Furthermore, the DFD highlights two distinct logical flows. The *Deploy* flow represents the management channel, carrying infrastructure configurations and execution instructions

from the Ansible Controller to the target nodes. The *Data Replication* flow represents the application-level synchronization of data moving from the Children to the central Root. Although the logical direction of data replication remains consistent across topologies, the network level initiation of these connections will vary depending on whether the Child-Push or Root-Pull architecture is implemented, as thoroughly discussed in Chapter 3.

4.1.2 Trust Boundaries

A critical outcome of the system modeling phase is the delineation of Trust Boundaries. A trust boundary represents a perimeter across which level of trust changes, typically separating highly trusted components from untrusted external networks or environments. Based on the security assumptions of this thesis, the Hierarchical ODA ecosystem is partitioned as follows:

Trusted Components (Out of Scope)

Components within this boundary are assumed to be secure by definition and are therefore excluded from the adversarial threat model:

Ansible Controller and Supply Chain : The administrative workstation running the Ansible engine, along with its associated Git and Docker repositories, is considered a trusted environment. No Threat Modeling is performed against the controller itself.

Applications Internals : As mentioned in Section 2.1.2, the entire baseline ODA stack (e.g., API Gateway, Data Pump, InfluxDB) is inherited as a pre-validated, encapsulated "black box". Consequently, the application-level logic, source code vulnerabilities, and the intra-node communications occurring within the local Docker network are explicitly out of scope. The threat model assumes these microservices operate securely within their isolated local boundaries, focusing the analysis entirely on the inter-node replication pipelines and the infrastructure orchestration.

Hardware and Hypervisor : The physical integrity of the host machines and the underlying virtualization layers are assumed to be uncompromised.

Infrastructure-Level Network Filtering : While exposing ports introduces the theoretical risk of low-level network DoS (e.g., TCP SYN floods), mitigating these infrastructure threats is outside the scope of this Zero Trust application architecture. It is assumed that basic ingress filtering and network availability are enforced by the underlying Cloud provider, external hardware firewalls, or the VPN overlay routing itself.

Untrusted Components (In Scope)

Components within this boundary represent the attack surface and are the primary focus of the threat model:

Deployed Nodes (Zero Trust): Once provisioned, both the Root and Child Virtual Machines are considered untrusted. Given their distributed nature, especially at the Edge, they are susceptible to local reconnaissance, unauthorized access, and malicious configuration drift.

Transport Network: The Data Replication flow traverses the public Internet, rendering the communication channel inherently insecure and exposed to interception and manipulation.

Management Channel: The Deploy flow, relying on SSH connections between the Controller and the target nodes, crosses an untrusted network and is exposed to potential network-level adversaries.

4.2 Threat Identification

After explicitly defining the system architecture and its trust boundaries, the next logical step in our framework is to answer the question: "What can go wrong?". This phase, known as Threat Identification, uncovers the potential vulnerabilities and attack vectors within the Hierarchical ODA environment.

While traditional methodologies like STRIDE are highly effective, they are inherently system-centric, focusing primarily on software components and data flows. Relying solely on a system-centric view can sometimes obscure the human and adversarial elements of cyber-threats. To address this limitation and achieve a more robust analysis, this thesis adopts a hybrid methodology, drawing inspiration from the *Hybrid Threat Modeling Method* (hTMM) [30]. The hTMM advocates for the integration of system-centric frameworks with attacker-centric techniques to generate a more comprehensive threat landscape.

4.2.1 Attacker Modeling

Building upon the attacker-centric dimension introduced by the hTMM, it is essential to step outside the abstract realm of system vulnerabilities and visualize concrete abuse cases by defining the adversaries. To generate and formalize these attacker models with academic and industrial rigor, we rely on the guidelines provided by the NIST, *Guide for Conducting Risk Assessments* [31]. This standard dictates that to accurately evaluate risks, one must characterize *Adversarial Threat Sources* based on three fundamental dimensions:

1. **Capability:** The resources, technical skills, and opportunities available to the adversary.
2. **Intent:** The ultimate goal or motivation driving the attack.
3. **Targeting:** The specific operational focus of the adversary's actions.

Based on the defined Trust Boundaries and the assumption that both the transport network and the deployed environments are untrusted, we have modeled three distinct adversarial profiles. These personas encapsulate the most critical threats to the Hierarchical ODA infrastructure, ranging from external network eavesdroppers to fully privileged compromised nodes.

The Network Adversary (External)

This profile represents an external threat actor located along the untrusted transport network that connects the geographically dispersed components. Possessing the technical means to passively intercept and actively manipulate traffic across the public network, this adversary controls or monitors intermediary network nodes without having direct access to the host machines.

Their primary intent is opportunistic eavesdropping to extract sensitive data or management instructions. Furthermore, they may attempt to execute Man-in-the-Middle (MitM) attacks or establish rogue connections to exposed endpoints to bypass authentication mechanisms.

Operationally, this adversary strictly targets the application-level Data Replication flow between the Kafka brokers, the SSH-based deployment channel originating from the Ansible Controller, and the public ingress ports of any exposed edge or core broker.

The Malicious Observer (Local Insider)

This profile represents an insider threat or an external attacker who has breached the initial perimeter but holds limited, unprivileged shell access to a Zero Trust node. Operating without root privileges, the adversary can navigate the local file system, inspect environment variables, and observe running processes, but lacks the permissions to modify core system configurations or alter container states.

The primary goal of this persona is local reconnaissance and secret harvesting. Specifically, the attacker seeks to pivot or escalate privileges by discovering poorly protected cryptographic materials, hardcoded credentials, or exposed configuration files.

Consequently, their targeting is confined to the local environment of the compromised machine, focusing on deployment artifacts left by the IaC orchestrator, such as Kafka certificates and sensitive local directories.

The Compromised Administrator (Node Takeover)

Representing the most severe threat profile, the Compromised Administrator simulates a worst-case scenario in which a node deployment is completely overtaken due to a critical vulnerability or local physical tampering. This adversary has achieved full administrative (root) privileges, granting them unrestricted control over the local operating system, the Docker daemon, and the local Kafka broker.

Driven by the intent to sabotage the local data pipeline, the attacker may intentionally cause configuration drift to evade security policies. Alternatively, they might leverage the hijacked node's trusted position to launch systemic attacks, such as application-level DoS or lateral movement, against the central core infrastructure or Edge nodes.

To achieve these goals, the attacker specifically targets the local system configurations, the MM2 replication governance logic, and the outbound network connections directed toward the healthy nodes.

4.2.2 STRIDE and MITRE Mapping

With the adversarial profiles established, we methodically map their actions against both the Child-Push and Root-Pull architectures. To provide a formal and standardized categorization, we evaluate the threats using the STRIDE categories combined with the technical identifiers from the MITRE ATT&CK framework.

Given that the impact of an adversary heavily depends on the underlying replication topology, the mapping explicitly distinguishes between the architectural contexts and the specific node compromised. The complete categorization of these attack vectors is summarized in Table 4.1

Threat ID	Adversary Profile	Attack Surface	Architecture	STRIDE Categories	MITRE Techniques
[T-NA.1]	Network Adversary	Deploy Flow	Both	Info Disclosure, Spoofing, Tampering	Network Sniffing, Adversary-in-the-Middle
[T-NA.2]	Network Adversary	Data Replication Flow	Child-Push	Info Disclosure, Spoofing, Tampering	Network Sniffing, Adversary-in-the-Middle
[T-NA.3]	Network Adversary	Data Replication Flow	Root-Pull	Info Disclosure, Spoofing, Tampering	Network Sniffing, Adversary-in-the-Middle
[T-MO.1]	Malicious Observer	Child Node	Child-Push	Information Disclosure	Unsecured Credentials, File and Directory Discovery
[T-MO.2]	Malicious Observer	Child Node	Root-Pull	Information Disclosure	Unsecured Credentials, File and Directory Discovery
[T-MO.3]	Malicious Observer	Root Node	Child-Push	Information Disclosure	Unsecured Credentials, File and Directory Discovery
[T-MO.4]	Malicious Observer	Root Node	Root-Pull	Information Disclosure	Unsecured Credentials, File and Directory Discovery
[T-CA.1]	Compromised Administrator	Child Node	Child-Push	Tampering, Spoofing, Denial of Service	Impair Defenses, Data Manipulation, Endpoint DoS
[T-CA.2]	Compromised Administrator	Child Node	Root-Pull	Tampering, Denial of Service	Impair Defenses, Data Destruction
[T-CA.3]	Compromised Administrator	Root Node	Child-Push	Tampering, Denial of Service	Impair Defenses, Data Destruction
[T-CA.4]	Compromised Administrator	Root Node	Root-Pull	Tampering, Spoofing, Denial of Service	Impair Defenses, Data Manipulation, Endpoint DoS

Table 4.1: Hierarchical ODA Threat Matrix mapping adversaries to architectures.

Mapping the Network Adversary

The Network Adversary operates exclusively on the untrusted transport network. While the management flow remains structurally identical across topologies, the Data Replication flow inverts the attack surface, dictating whether the central core or the peripheral edge is exposed to the public network.

Starting with the management channel, the adversary passively intercepts or actively manipulates the SSH connections from the Ansible Controller to the target nodes [T-NA.1]. This allows the extraction of unencrypted IaC artifacts or the execution of SSH downgrade attacks to alter configuration instructions in transit.

Regarding the Data Replication flow, the chosen architecture dictates the attacker's methodology. In the Child-Push architecture, the adversary targets the publicly exposed Root broker [T-NA.2]. Actions include sniffing unencrypted data, spoofing Edge nodes to establish unauthorized sessions, or altering legitimate data via MitM attacks. Conversely, in the Root-Pull architecture, the adversary targets the distributed Edge brokers [T-NA.3]. Here, the attacker can spoof the Root node to scrape sensitive data directly from the Children, alter the pulled data in transit, or attempt unauthorized connections to the Edge environment.

Mapping the Malicious Observer

Unlike the Compromised Administrator, the Malicious Observer lacks the privileges to alter the system state or disrupt services directly. Instead, their primary objective is Information Disclosure through secret harvesting. The architectural choice fundamentally dictates where the most sensitive artifacts are stored, specifically the MM2 credentials and replication certificates, thus altering the risk profile of each node.

When targeting a Child node in the Child-Push architecture [T-MO.1], the attacker navigates the local environment to find Ansible deployment artifacts or misconfigured files containing the MM2 configurations. Because MM2 runs locally in this topology, the attacker can harvest the replication credentials and certificates required to connect to the Root. Extracting these secrets allows the adversary to pivot and authenticate against the central Root broker from an external machine, escalating a local edge breach into a systemic core compromise. In contrast, if the compromised Child node operates in a Root-Pull architecture [T-MO.2], the attacker performs local reconnaissance and can successfully harvest local Kafka certificates. However, because the MM2 replication engine is centralized on the Root, this edge node contains no outbound replication credentials. The blast radius remains strictly confined to the local node, effectively preventing credential-based lateral movement toward the central infrastructure.

Shifting to the core infrastructure, if the attacker targets the Root server's local environment in a Child-Push topology [T-MO.3], they cannot extract outbound replication credentials to move laterally back to the Edge, since the MM2 replication engines run on the peripheral Child nodes. However, local reconnaissance successfully reveals Root broker administrative credentials and centralized deployment artifacts, resulting in a potentially localized data exposure. Finally, in the Root-Pull topology, the impact of local reconnaissance on the core is catastrophic [T-MO.4]. By exploring the local file system, the attacker gains access to the centralized MM2 configuration directories. These unencrypted IaC artifacts contain the connection secrets and certificates for the entire fleet of distributed Child nodes, actively enabling widespread credential-based lateral movement back to the Edge.

Mapping the Compromised Administrator

The Compromised Administrator represents a catastrophic local failure. However, the systemic blast radius of this compromise is strictly dictated by the chosen architecture. In the Child-Push model, a compromised Edge node poses a critical threat to the core infrastructure because the replication governance is decentralized. Instead, in the Root-Pull model, a compromised Edge is isolated, but a Root compromise is devastating.

Focusing on the Edge, a compromised Child node in the Child-Push architecture [T-CA.1] means the attacker completely controls the MM2 instance, as the replication governance resides locally. By causing deliberate configuration drift (modifying local MM2 configs or Docker deployments), the adversary can arbitrarily alter the topics being pushed, inject fabricated data directly into the Root broker, or attempt a tenant impersonation attack. Furthermore, they can exhaust the Root's resources via an application-level Denial of Service (flooding the core with legitimate but overwhelming data records). Conversely, in the Root-Pull architecture, the attacker has full control over the local node and can sabotage the local data pipeline [T-CA.2]. However, because the MM2 engine is centralized on the Root, the Edge node acts purely as a passive server. The attacker cannot initiate connections to spoof other tenants or force arbitrary data into the core infrastructure, effectively quarantining the breach and preventing lateral movement toward the Cloud.

At the central level, a root compromise of the Root node in a Child-Push architecture [T-CA.3] is inherently fatal to the global data pipeline. The attacker can wipe or manipulate the aggregated database, destroy the Root Kafka broker, or exhaust core resources. However, because the replication governance is decentralized and the Root acts only as a receiver, the adversary cannot directly leverage the core to push malicious synchronization configurations back to the healthy Child nodes. On the other hand, a Root node compro-

mise in the Root-Pull architecture [T-CA.4] represents the absolute worst-case systemic failure. Beyond wiping the aggregated database, the attacker gains absolute control over the centralized MM2 engines. This allows the adversary to maliciously alter the synchronization logic, spoof identities, and manipulate data ingestion pipelines for all connected Child nodes simultaneously, effectively taking over the entire hierarchical infrastructure.

4.3 Response and Mitigations

Having identified the threats across both architectures, the final phase of our threat modeling framework starts to address the question "What are we going to do about it?". According to standard risk management methodologies, an organization can choose to accept, transfer, avoid, or mitigate a risk. Given the inherently untrusted nature of the deployment environments and the critical role of the data pipeline, this thesis adopts a mitigation strategy. Specifically, our objective is to actively mitigate all identified threats to the greatest extent practically possible, prioritizing automated security enforcement over risk acceptance.

In distributed, scaling Edge-to-Cloud ecosystems, manual security configurations are error-prone and ultimately unsustainable. Therefore, our response strategy does not rely on static guidelines but leverages Ansible as a continuous Security-as-Code engine. By enforcing security policies dynamically through IaC, we aim to reduce the likelihood and impact of the identified threats to an acceptable level.

The following actionable mitigations form the strategic baseline that will be practically implemented and validated in the Proof of Concept (Chapter 5) and subsequent results validation (Chapter 6). Each countermeasure is mapped to the specific Threat IDs defined in Table 4.1.

4.3.1 Transport Security and Cryptographic Enforcement

To protect the communication channels traversing the untrusted network, the infrastructure must completely deprecate plaintext protocols in favor of robust, mutually authenticated connections across both the Deploy flow and Data Replication flow.

Starting with the management channel, the architecture enforces *Strict SSH Host Key Verification*. Before transmitting any IaC instruction, the Ansible Controller authenticates the identity of the target nodes against a pre-validated set of cryptographic host keys (stored on the Controller). This proactive validation immediately drops any intercepted or spoofed connection attempt, effectively mitigating Network Sniffing and the Adversary-in-the-Middle threats against the Deploy flow ([T-NA.1]).

Alongside the management channel, the application data flow is protected by implementing *mTLS for Kafka Brokers*. Specifically, Ansible is used to provision, distribute, and enforce strictly authenticated x509 certificates for all external Kafka listeners and MM2 clients. This ensures that only authorized nodes can interact with the exposed brokers, encrypting the payload in transit and preventing unauthorized data injection. Consequently, this cryptographic boundary directly mitigates the threats posed by the Network Adversary on the Replication Flow ([T-NA.2], [T-NA.3]), effectively neutralizing Network Sniffing and Adversary-in-the-Middle attempts.

4.3.2 Secret Management and File System Hardening

To counter the threat of local reconnaissance and secret harvesting on Zero Trust nodes, the deployment artifacts must be rigorously protected at rest.

This is achieved through the integration of *Ansible Vault and strict POSIX permissions*. Sensitive IaC variables (e.g., MM2 credentials, keystore passwords) are encrypted within the Ansible Controller using Ansible Vault, protecting the Deploy flow artifacts from network interception ([T-NA.1]). During deployment, these secrets are decrypted on-the-fly into the target machine's memory and materialized into files enforced with strict POSIX boundaries. By restricting file access at the Operating System (OS) level, this strategy effectively blinds the Malicious Observer ([T-MO.1] through [T-MO.4]), preventing File and Directory Discovery and blocking credential-based lateral movement.

4.3.3 Replication Governance and Drift Control

Addressing the Compromised Administrator requires containing the blast radius of a root takeover and ensuring the core infrastructure cannot be destroyed by a rogue node.

Rather than granting blanket access to the replication engines, the system enforces *Kafka ACLs and Quotas*. Ansible dynamically configures access control lists based on the principle of Least Privilege, ensuring, for instance, that a Child node's MM2 process only possesses write permissions for its specific topics. Coupled with network Quotas to cap the bandwidth of individual clients, this governance model prevents arbitrary Data Manipulations, Spoofing, and mitigates volumetric Endpoint Denial of Service ([T-CA.1], [T-CA.4]).

Furthermore, to counter intentional configuration drift caused by a local node takeover, the architecture introduces *Drift Control via Ansible*. By scheduling the Ansible orchestration pipeline to run continuously (e.g., via a cron job) and leveraging Ansible's intrinsic idempotency, the system actively enforces its desired state. Any unauthorized modification to the MM2 local configurations is automatically detected and overwritten with the

trusted baseline. This self-healing mechanism actively remediates Impaired Defenses and malicious administrative tampering ([**T-CA.1**] through [**T-CA.4**]).

Chapter 5

Proof of Concept Implementation

Following the theoretical analysis and the Threat Modeling conducted in Chapter 4, this chapter presents the practical realization of the Secure by Design Hierarchical ODA infrastructures. The primary objective of this PoC is to demonstrate how the identified vulnerabilities can be actively mitigated through an automated, declarative approach using Ansible as a Security-as-Code engine.

This chapter outlines the transition from architectural design to deployment. First, it details the testbed environment and the physical configuration of the target nodes. Subsequently, it illustrates the structural organization of the IaC code. Finally, it provides a deep dive into the implementation of security countermeasures, directly addressing the adversarial profiles introduced in Section 4.2.1 and the specific Threat IDs modeled in Section 4.2.2 for both Child-Push and Root-Pull topologies.

5.1 Testbed Environment

To simulate a distributed hierarchical ecosystem, the PoC was deployed on a realistic, multi-node virtualized infrastructure provided by the *Sistema Informatico Dipartimentale* of the *University of Pisa*. This setup allowed for the practical evaluation of both replication topologies across logically separated environments, adhering to the trust boundaries defined in Section 4.1.2.

The deployment orchestration is driven from a dedicated *Fedora*¹ Windows Subsystem for Linux (WSL) installation running on a local machine. This environment acts as the trusted Ansible Controller, handling all management operations, secure secret generation via Ansible Vault, and the enforcement of the desired infrastructure state.

Conversely, the application layer is hosted on three distinct Virtual Machines (VMs)

¹Specific Fedora version used is Fedora Linux 42.

running an *Ubuntu* Long-Term Support (LTS)² distribution. Consistent with the Zero Trust paradigm, these target nodes are treated as untrusted environments. Specifically, the infrastructure is partitioned into one Root node, acting as the centralized Cloud core, and two Child nodes, acting as the peripheral Edge instances.

All communications, including the deployment traffic originating from the Controller and the data replication streams between the Root and Child nodes, are routed through the University's VPN. This architectural scenario emulates the characteristics of a WAN, introducing real-world network latency and providing a realistic baseline to evaluate the transport security mechanisms against potential Network Adversaries.

5.1.1 Virtual Machines Specifications

To ensure a reliable deployment of the hierarchical ODA ecosystem, the underlying hardware must provide adequate compute capacity. The containerized stack demands specific baseline resources to prevent memory exhaustion and scheduling bottlenecks during data ingestion.

Consequently, each virtual machine is provisioned with two virtual CPUs (Intel Xeon Gold 5120 @ 2.20 GHz), up to 8 GB of dynamically allocated memory, and 38 GB of storage space.

5.1.2 SSH Configuration

Prior to the automated deployment, the SSH configuration on the Controller was explicitly hardened to stabilize the connection across the VPN and enforce public-key authentication, ensuring a secure management channel (Deploy Flow) out-of-the-box.

To achieve this, an *ED25519 elliptic curve* key pair was generated on the trusted Ansible Controller. Relying on the *Trusted Computing Base* assumption established in Chapter 4, the private key was generated without a passphrase³ to enable fully non-interactive automation. The public key was then distributed to the `~/.ssh/authorized_keys` file of the dedicated deployment user on the target VMs.

Furthermore, to maintain a clean separation of concerns, the SSH transport layer is managed entirely by the *OpenSSH* client configuration (`~/.ssh/config`) on the Controller. This approach decouples the cryptographic identity management, strict host key verification, and connection stabilization parameters (such as `ServerAliveInterval` for VPN persistence) from the Ansible declarative logic and allows the IaC orchestrator to rely completely on this underlying secure transport to communicate with the remote instances.

²Specific Ubuntu version used is 22.04.5 LTS.

³A viable alternative is to adopt a passphrase in conjunction with an `ssh-agent` to manage interactive authentication prompts during automated runs.

The procedural details regarding the generation and distribution of the cryptographic keys, alongside the complete OpenSSH client configuration file used to stabilize the VPN connections and prevent Adversary-in-the-Middle attacks, are provided in Appendix A.1.

5.1.3 Certificates Generation

Alongside the management channel, the application data flow (Replication Flow) relies on mTLS to establish secure communication between endpoints. To avoid manual misconfigurations and ensure that each node receives a uniquely identifiable certificate, a dedicated *Bash* script was developed for each topology to automate the internal Public Key Infrastructure (PKI) generation.

For the Child-Push topology, the principle of least privilege was applied directly at the cryptographic asset level: the Root Kafka broker acts as a Server, while the MM2 instances on the Child nodes act exclusively as Clients. A key aspect of this setup is that the *Common Name* (CN) assigned to each certificate is programmatically mapped to the exact `inventory_hostname` defined in the Ansible repository (e.g., `oda_child_1`). This naming consistency serves as the basis for the dynamic ACLs evaluated in the subsequent execution phases.

To briefly clarify the cryptographic mechanics implemented in the proposed automation, the PKI relies on three standard components:

- **Certificate Authority (CA):** Acts as the internal root of trust, responsible for signing all certificates within the ecosystem.
- **Keystore:** Contains the node's private key and its CA-signed public certificate, essentially serving as the node's cryptographic identity card to prove who it is.
- **Truststore:** Holds the public certificate of the CA. It is used by the node to verify that the certificates presented by other connecting parties are authentic and issued by the same trusted authority.

Listing 5.1 presents the PKI automation script tailored for the Child-Push architecture. For the alternative Root-Pull topology, where the Root node actively fetches data and acts as a client to the Child servers, the cryptographic roles are logically inverted. The modified PKI generation script, adapted to accommodate this reversed trust model, is provided for reference in Appendix A.2.

```
1 #!/bin/bash
2
3 PASSWORD="XXX"
4 ROOT_CN="oda_root"
```

```

5 ROOT_IP="XXX.XXX.XXX.XXX"
6 CHILD_NODES=("oda_child_1" "oda_child_2") # use ansible inventory
   naming
7 OUT_DIR="oda_certs_output"
8
9 echo "starting ODA mTLS certificates generation "
10
11 rm -rf $OUT_DIR
12 mkdir -p $OUT_DIR/ca
13 cd $OUT_DIR
14
15 # 1. CA (Certificate Authority)
16 echo "generating CA"
17 openssl req -new -newkey rsa:4096 -days 3650 -x509 \
18   -subj "/CN=ODA-Internal-CA" \
19   -keyout ca/ca-key -out ca/ca-cert -nodes > /dev/null 2>&1
20
21 #
22 # 2. ROOT BROKER
23 #
24 echo "generating keystore and truststore server for root broker"
25
26 mkdir -p $ROOT_CN
27
28 # Truststore
29 keytool -keystore $ROOT_CN/kafka.server.truststore.jks -alias CARoot -
   import -file ca/ca-cert -storepass $PASSWORD -noprompt > /dev/null
   2>&1
30
31 # Keystore and Signing
32 keytool -genkey -keystore $ROOT_CN/kafka.server.keystore.jks -alias
   server -dname "CN=$ROOT_CN" \
33   -keyalg RSA -storepass $PASSWORD -keypass $PASSWORD \
34   -ext SAN=DNS:localhost,IP:127.0.0.1,IP:0.0.0.0,IP:$ROOT_IP > /dev/
   null 2>&1
35
36 keytool -keystore $ROOT_CN/kafka.server.keystore.jks -alias server -
   certreq -file $ROOT_CN/server-req -storepass $PASSWORD > /dev/null
   2>&1
37
38 echo "subjectAltName=DNS:localhost,IP:127.0.0.1,IP:0.0.0.0,IP:$ROOT_IP"
   > $ROOT_CN/extfile.cnf
39
40 openssl x509 -req -CA ca/ca-cert -CAkey ca/ca-key -in $ROOT_CN/server-
   req -out $ROOT_CN/server-cert -days 365 -CAcreateserial -passin
   pass:$PASSWORD -extfile $ROOT_CN/extfile.cnf > /dev/null 2>&1
41
42 keytool -keystore $ROOT_CN/kafka.server.keystore.jks -alias CARoot -
   import -file ca/ca-cert -storepass $PASSWORD -noprompt > /dev/null
   2>&1
43 keytool -keystore $ROOT_CN/kafka.server.keystore.jks -alias server -
   import -file $ROOT_CN/server-cert -storepass $PASSWORD -noprompt >
   /dev/null 2>&1
44
45 #
46 # 3. CHILD NODES

```

```

47 #
48 for CHILD in "${CHILD_NODES[@]}"; do
49     echo "generating keystore and truststore client for $CHILD"
50     mkdir -p $CHILD
51
52     # Truststore
53     keytool -keystore $CHILD/kafka.client.truststore.jks -alias CARoot -
        import -file ca/ca-cert -storepass $PASSWORD -noprompt > /dev/null
        2>&1
54
55     # Keystore and Sign (CN matches ansible naming)
56     keytool -genkey -keystore $CHILD/kafka.client.keystore.jks -alias
        client -dname "CN=$CHILD" \
57         -keyalg RSA -storepass $PASSWORD -keypass $PASSWORD > /dev/null
        2>&1
58     keytool -keystore $CHILD/kafka.client.keystore.jks -alias client -
        certreq -file $CHILD/client-req -storepass $PASSWORD > /dev/null
        2>&1
59     openssl x509 -req -CA ca/ca-cert -CAkey ca/ca-key -in $CHILD/client-
        req -out $CHILD/client-cert -days 365 -CAcreateserial -passin pass:
        $PASSWORD > /dev/null 2>&1
60     keytool -keystore $CHILD/kafka.client.keystore.jks -alias CARoot -
        import -file ca/ca-cert -storepass $PASSWORD -noprompt > /dev/null
        2>&1
61     keytool -keystore $CHILD/kafka.client.keystore.jks -alias client -
        import -file $CHILD/client-cert -storepass $PASSWORD -noprompt > /
        dev/null 2>&1
62 done
63
64 # Cleaning
65 echo "cleaning csr and temporary files"
66 find . -name "*-req" -delete
67 find . -name "*-cert" -delete
68 find . -name "*.srl" -delete
69 find . -name "*.cnf" -delete
70
71 echo "finished! Output in '$OUT_DIR'"

```

Listing 5.1: Automated PKI generation script for the Child-Push topology.

5.2 Infrastructure as Code Structure

The transition from a localized environment to a distributed Edge-to-Cloud architecture required a robust, automated deployment strategy. To achieve this, the ODA deployment pipeline was refactored using Ansible, adopting an IaC and Security-as-Code paradigm.

The primary objective of this structural reorganization was to create a modular, idempotent, and secure deployment engine capable of dynamically configuring both the Child-Push and Root-Pull topologies, minimizing manual intervention on the Target Nodes.

5.2.1 Ansible Repository Architecture

To maintain a clean, scalable, and secure codebase, the Ansible repository was structured following established DevOps practices. A fundamental design choice was the enforcement of a clear separation of concerns among configuration data, cryptographic assets, execution logic, and deployment targets.

The complete hierarchical structure of the deployment repository, common to both topologies, is illustrated in Listing 5.2.

```
ansible/
|-- ansible.cfg
|-- files/
|   |-- oda_base/
|   \-- secrets/
|       |-- oda_child_1/
|       |-- oda_child_2/
|       \-- oda_root/
|-- group_vars/
|   |-- all.yml
|   |-- child.yml
|   \-- root.yml
|-- inventory/
|   \-- hosts.yml
|-- playbooks/
|   |-- deploy.yml
|   |-- start.yml
|   \-- stop.yml
|-- roles/
|   |-- oda_child/
|   \-- oda_root/
|-- templates/
|   |-- compose_child.yml.j2
|   |-- compose_root.yml.j2
|   |-- env.j2
|   \-- root.properties.j2
\-- vault/
    \-- .vault_pass
```

Listing 5.2: Directory tree summary of the Ansible repository, common to both Hierarchical ODA topologies.

The directory tree is logically partitioned into distinct functional domains. Each directory serves a specific purpose in the declarative pipeline, ensuring that the deployment is both modular and auditable:

- `ansible.cfg`: is the principal configuration file that dictates the behavior of the

Ansible Controller. It applies critical optimization, such as enabling SSH pipelining to accelerate task execution over the VPN, and sets default path values.

- **files/:** serves as the repository for static assets and cryptographic materials. Notably, it contains the **secrets/** subdirectory, which stores the pre-generated mTLS *Keystores* and *Truststores*. By strictly isolating these certificates into subdirectories named after their respective Target Nodes, the pipeline programmatically enforces the Least Privilege principle, transferring only the strictly necessary keys to each VM. The directory also houses the **oda_base** folder containing the static source code of the microservices.
- **group_vars/:** centralizes the declarative state of the infrastructure. The **all.yml** file defines global parameters (e.g., standard ports and base deployment paths), while **root.yml** and **child.yml** contain node-specific variables. This separation avoids hardcoding values in the execution logic, enabling the dynamic resolution of elements (such as MM2 configuration values) based on the target host's logical group.
- **inventory/:** contains the **hosts.yml** file, which maps the logical node groups (Root and Child) to the actual VPN IP addresses of the target machines. This abstraction is fundamental as it completely decouples the deployment logic from the physical network topology, allowing the same codebase to target different environments with minimum modifications.
- **playbooks/:** contains the high-level orchestration entries **deploy.yml**, **start.yml**, **stop.yml**. These files map inventory groups to specific roles or operational phases, cleanly separating the static provisioning of the infrastructure from the actual container runtime execution.
- **roles/:** encapsulates the core execution logic into the distinct, self-contained entities **oda_root** and **oda_child**. Each role defines the atomic tasks required to transition the target node from the baseline state to the desired configuration. This includes creating hardened directories, deploying configuration files, and applying runtime security measures as dynamic ACLs.
- **templates/:** hosts the dynamic *Jinja2* (.j2) templates, such as Docker Compose definitions, environment variables (.env), and the core configuration files for MM2. Specifically, it includes **root.properties** for the Child-Push topology and its architectural twin **child.properties.j2** in the Root-Pull scenario. Instead of transferring static files, Ansible compiles these templates at runtime, injecting

the variables resolved from `group_vars` to produce customized, state-enforcing configurations for each specific node.

- `vault/`: securely isolates the decryption key (`.vault_pass`) used by Ansible Vault. This ensures that sensitive parameters, such as the cryptographic passwords, remain strictly encrypted at rest on the Controller and are explicitly excluded from version control, preventing accidental secret leakage.

5.2.2 Playbook Orchestration

The high-level orchestration of the environment is driven by purpose-built playbooks. Because Ansible is an agent-less automation engine, these playbooks interact with the nodes directly over the SSH transport layer, eliminating the need for additional daemon installations on the target environments.

A core architectural decision in this PoC was the implementation of a *Lifecycle Separation* pattern, as described by the State Machine Diagram in Figure 5.1. Rather than combining state provisioning and application execution into a single monolithic execution run, the deployment lifecycle is intentionally divided into distinct, isolated phases. This modular approach ensures that the ODA ecosystem is brought online in a controlled and repeatable manner.

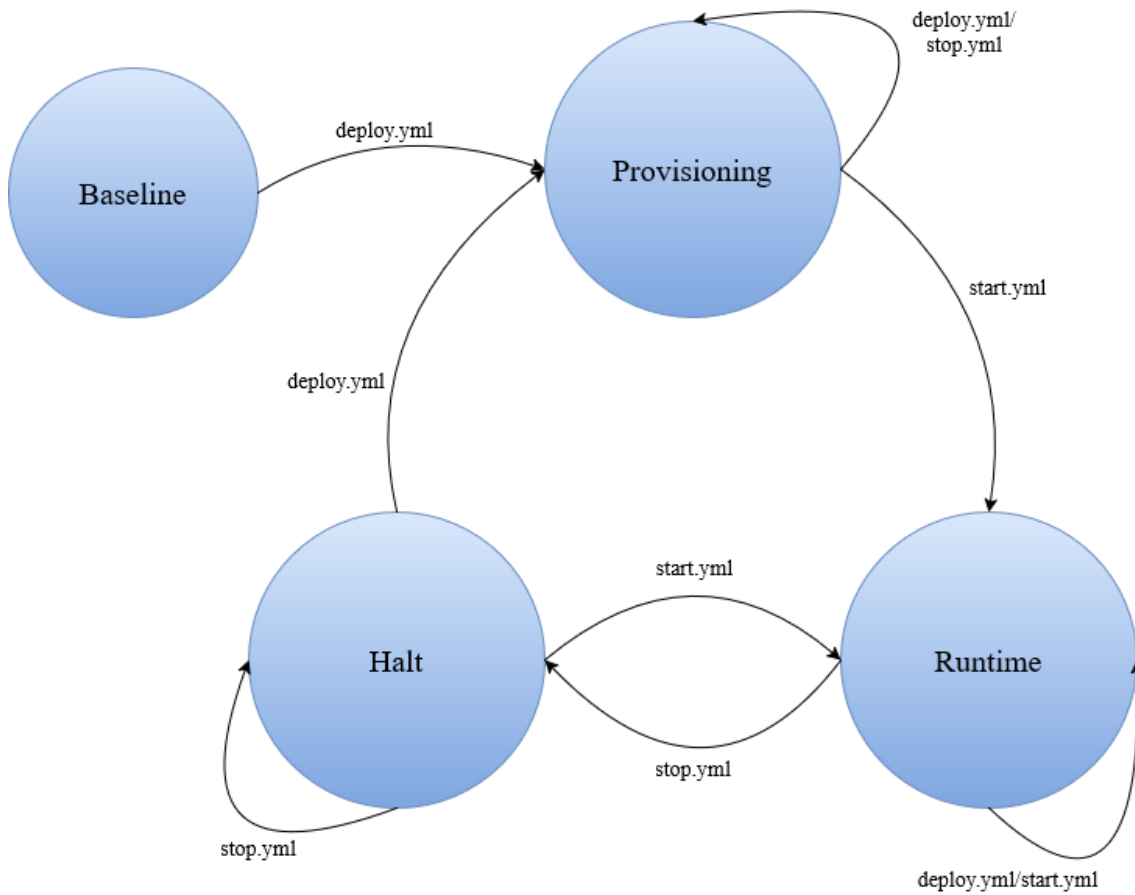


Figure 5.1: State Machine Diagram illustrating the infrastructure lifecycle and transitions.

The orchestration relies primarily on three playbooks, each mapping to a specific operational state. The `deploy.yml` playbook handles the logical *Provisioning Phase*, acting as the static configuration engine. It maps the `root` and `child` inventory groups to their respective execution roles (`oda_root` and `oda_child`). The associated tasks focus on system scaffolding: creating directories, securely distributing the mTLS certificates, transferring static code, and rendering the dynamic Jinja2 templates. Importantly, this playbook is designed to be idempotent and halts before any container execution occurs. As indicated by the self-loops in the state diagram, executing this playbook multiple times can gracefully reconcile configuration drifts without disrupting the active system state. The command required to trigger this phase is provided in Listing 5.3, which includes the necessary vault password file to resolve encrypted variables.

```
$ ansible-playbook -i inventory/hosts.yml playbooks/deploy.yml  
--vault-password-file vault/.vault_pass
```

Listing 5.3: Ansible Playbook command to complete the Provisioning Phase.

Once the infrastructure is provisioned, the `start.yml` playbook manages the *Runtime Phase*, which is dedicated solely to the operationalization of the stack. It first triggers the container engines across all nodes via Docker Compose to build and start the ODA microservices. Subsequently, it handles post-start runtime configurations. Once the Kafka broker is initialized, Ansible dynamically uses the `include_role` directive to inject security configurations, such as dynamic ACLs and network quotas, directly into the running containers. The command to execute this phase is shown in Listing 5.4.

```
$ ansible-playbook -i inventory/hosts.yml playbooks/start.yml
```

Listing 5.4: Ansible Playbook command to complete the Runtime Phase.

Finally, the `stop.yml` playbook enables the *Halt Phase*. It functions as a utility script designed to gracefully stop the active containers across all environments without tearing down the underlying infrastructure. By leveraging the `state: stopped` directive, it preserves the existing volumes, generated configurations, and application states, allowing for a safe, temporary suspension of the hierarchical ecosystem, as demonstrated in Listing 5.5.

```
$ ansible-playbook -i inventory/hosts.yml playbooks/stop.yml
```

Listing 5.5: Ansible Playbook command to execute the Halt Phase.

5.3 Implementing Security-as-Code Mitigations

This section bridges the theoretical Threat Model defined in Chapter 4 with the practical IaC deployment. It details how the proposed strategic countermeasures were technically implemented using Ansible. By translating security policies into declarative code, the infrastructure autonomously enforces the desired security state across both the Child-Push and Root-Pull topologies, effectively neutralizing the modeled adversarial profiles.

5.3.1 Transport Security and Cryptographic Enforcement Implementation

As established in the Threat Model, the Data Replication Flow traversing the VPN is highly susceptible to Network Adversaries attempting eavesdropping or MitM attacks. To actively mitigate these threats (specifically [T-NA.2] and [T-NA.3]), the infrastructure mandates mTLS authentication for all inter-node communications. Obviously, beyond

mutually authenticating the identities of the interacting actors, this mTLS implementation is responsible for establishing a secure TLS tunnel. This ensures that the entire communication channel is encrypted, guaranteeing both the confidentiality and integrity of the application payloads in transit.

While the automated PKI generation scripts successfully create the required cryptographic assets on the Controller, the actual security enforcement relies on Ansible safely distributing and binding these certificates to the correct isolated containers.

Child-Push Scenario

In the Child-Push architecture, the Root node acts as the central server exposing its Kafka broker, while the distributed MM2 instances on the Child nodes act as active clients.

The Ansible implementation natively reflects this Trust Model. During the Provisioning Phase (`deploy.yml`), the `oda_child` role dynamically fetches the Client Keystore and Truststore using the `{{ inventory_hostname }}` variable, ensuring each node receives only its designated cryptographic identity. Once the certificates are safely copied to the target file system, Ansible orchestrates the bridging between the host machine and the Docker environment.

As shown in Listing 5.6, the certificates are injected into the MM2 container via Docker Compose volume mapping. The directory is mounted as read-only to prevent any runtime modification by a potentially compromised container. Concurrently, the MM2 replication logic is templated to enforce the TLS protocol for the outbound connection toward the Root.

```

1 # 1. Docker Compose Volume Mount (compose_child.yml.j2)
2   mirrmaker:
3     container_name: mirrmaker_push
4     volumes:
5       - {{ oda_deploy_path }}/secrets:{{ kafka_secrets_dir }}:ro
6
7 # 2. MirrorMaker 2 Templated Configuration (root.properties.j2)
8 ## root SSL config
9 root.security.protocol=SSL
10 root.ssl.truststore.location={{ kafka_secrets_dir }}/kafka.
    client.truststore.jks
11 root.ssl.keystore.location={{ kafka_secrets_dir }}/kafka.client.
    keystore.jks

```

Listing 5.6: Injection of cryptographic assets and TLS enforcement for the MM2 Client (Child Node).

Symmetrically, on the central Root node, the Kafka broker must be configured to

reject unencrypted or unauthenticated ingress traffic. Ansible achieves this by templating the broker's execution command to map the externally exposed endpoint to the TLS protocol, while simultaneously enforcing client certificate validation via the `ssl.client.auth=required` parameter.

This declarative pipeline effectively neutralizes the Network Adversary. Any attempt to connect to the Root broker without a valid, internal CA-signed client certificate matching the `inventory_hostname` is cryptographically rejected at the TLS handshake level, long before the application layer evaluates any ACL.

Root-Pull Scenario

In contrast to the Child-Push model, the Root-Pull topology inverts the data flow and the associated cryptographic Trust Model. The central Root node acts as the active network client pulling data, meaning it does not need to expose its local Kafka broker to the untrusted transport network. Conversely, the peripheral Child nodes act as passive servers, exposing their brokers to the Root.

To secure this inverted topology, the architecture must enforce TLS encryption and client authentication on the distributed Edge brokers. During the Provisioning Phase, Ansible distributes the Server Keystores to the Child nodes. Simultaneously, the Docker Compose template is configured to bind the exposed broker to the TLS protocol and require client authentication, neutralizing unauthorized ingress traffic at the edge.

On the central Root node, the complexity increases significantly. To preserve the principle of Least Privilege, a unique client certificate is generated for each specific MM2 instance residing on the Root. Ansible manages this 1-to-N distribution dynamically by iterating over the `groups['child']` inventory variables.

The true strength of this declarative approach manifests during the runtime templating. The Root's Docker Compose file employs a Jinja2 `{% for %}` loop to spawn a dedicated MM2 container for each target Child. To ensure absolute cryptographic isolation, each container mounts only its specific secret subdirectory as a read-only volume. Finally, the templated MM2 configuration enforces the TLS protocol for the outbound network connection targeting the remote Child broker, establishing the secure mTLS channel. A code snippet resembling this process is provided in Listing [5.7](#)

```

1 # 1. Dynamic MirrorMaker 2 Generation (compose_root.yml.j2)
2 {% for child in groups['child'] %}
3   mirrormaker_{{ child }}:
4     container_name: mirrormaker_{{ child }}_pull
5     volumes:
6       - {{ oda_deploy_path }}/secrets/{{ child }}:{{
7         kafka_secrets_dir }}:ro
8   {% endfor %}
9
10 # 2. MM2 mTLS Client Configuration (child.properties)
11 ## child SSL config
12 {{ item }}.security.protocol=SSL
13 {{ item }}.ssl.truststore.location={{ kafka_secrets_dir }}/kafka.
14   .client.truststore.jks
15 {{ item }}.ssl.keystore.location={{ kafka_secrets_dir }}/kafka.
16   .client.keystore.jks

```

Listing 5.7: Dynamic Container Generation and TLS Client Enforcement (Root Node).

5.3.2 Secret Management and File System Hardening Implementation

The distributed nature of Edge-to-Cloud architectures inherently exposes the deployed VMs to local reconnaissance. As modeled in Section 4.2.2, a Malicious Observer who gains unprivileged shell access to a Zero Trust node will actively attempt to harvest unsecured credentials and cryptographic materials to escalate privileges or move laterally across the infrastructure.

To neutralize these Information Disclosure threats (specifically [T-MO.1] through [T-MO.4]), the implementation relies on a two-tiered defense strategy: encrypting secrets at rest on the orchestration controller and hardening the file system permissions on the target nodes.

First, critical sensitive parameters are encrypted using Ansible Vault. In this Proof of Concept, Vault is utilized to protect `kafka_ssl_password`, a unified secret used for operational convenience to unlock the Keystores (`ssl.keystore.password`), the Truststores (`ssl.truststore.password`), and to access the actual private keys within the keystores (`ssl.key.password`). While this demonstrates the foundational Secret Management workflow, Ansible Vault's capabilities in a full production environment extend far beyond single-string encryption. It can natively encrypt entire configuration files, protect raw binary assets (such as the `.jks` files themselves), and manage multiple distinct vault identities for different environments.

Secondly, to protect the materialized artifacts on the untrusted nodes, Ansible enforces strict POSIX permissions, leveraging the architectural traits of the Docker engine.

Child-Push Scenario

In the Child-Push architecture, the peripheral Child node contains the outbound MM2 replication settings and credentials required to authenticate against the central Root. If a Malicious Observer harvests these secrets ([T-MO.1]), they can pivot and compromise the core infrastructure from an external location. Symmetrically, the Root node holds central administrative credentials that must be protected from local exposure ([T-MO.3]).

To mitigate this, the Ansible roles (`oda_child` and `oda_root`) are designed to scaffold the deployment environment using a *Default Deny* file system policy. As demonstrated in Listing 5.8, the very first task executed by the playbook is the creation of the base deployment directory with `0700` (`rwX-----`) permissions. This specific UNIX permission setup is a foundational security control. By restricting the base directory (`oda_deploy_path`) exclusively to the deployment user, the operating system blocks any unprivileged local user from traversing into the directory or listing its contents, regardless of the permissive modes (like `0644` or `0755`) applied to the inner files and subdirectories.

```
1 - name: create oda base deployment directory
2   file:
3     path: "{{ oda_deploy_path }}"
4     state: directory
5     mode: '0700'
6
7 - name: create secrets subdirectory, open to container
8   file:
9     path: "{{ oda_deploy_path }}/secrets"
10    state: directory
11    mode: '0755'
12
13 - name: generate oda env file
14   template:
15     src: ../../templates/env.j2
16     dest: "{{ oda_deploy_path }}/{{ env_filename }}"
17     mode: '0644'
```

Listing 5.8: Ansible tasks enforcing File System Hardening on the target node.

Crucially, this hardening does not disrupt the application runtime. Because the Docker daemon operates with root privileges, it bypasses the local `0700` traversal restriction. When the Docker Compose stack is initialized, the daemon successfully accesses the protected templates and can securely mount the required files and subdirectories into the isolated

containers. This implementation achieves robust credential protection at rest without compromising the operational requirements of the containerized services.

Root-Pull Scenario

In the Root-Pull architecture, the risk profile shifts toward the central node. As defined in the Threat Model ([T-MO.4]), a local breach on the Root node represents a catastrophic point of failure. If a Malicious Observer gains access to the centralized MM2 configuration directories, they could harvest the connection secrets for the entire fleet of distributed Edge nodes, enabling widespread credential-based lateral movement. Symmetrically, the peripheral Child nodes lack outbound replication credentials ([T-MO.2]), confining their local risk strictly to their own inbound server certificates.

While the `0700` Default Deny policy on the base deployment directory remains the primary shield against local unprivileged traversal, the Root node introduces a unique Secret Management complexity: it must securely materialize multiple distinct MM2 configuration files, each requiring the Vault-decrypted passwords.

Instead of deploying a single, static configuration file, the Ansible `oda_root` role leverages dynamic templating combined with a `loop` directive. As shown in Listing 5.9, Ansible iterates over the `groups['child']` inventory to generate an independent `.properties` file used to establish the MM2 replication pipeline for each Edge node. During this exact generation step, the Ansible Vault seamlessly injects the decrypted `kafka_ssl_password` directly into the specific SSL configuration blocks of the Jinja2 template.

```

1 # 1. Ansible Task: Generating the MM2 config file for every MM2
  instance corresponding to its Child (configure_mm2.yml)
2 - name: generate mm2 config files for each child
3   template:
4     src: ../../templates/child.properties.j2
5     dest: "{{ oda_deploy_path }}/config/mirrormaker/{{ item }}.
      properties"
6     mode: '0644'
7     force: yes
8   loop: "{{ groups['child'] }}"
9
10 # 2. Jinja2 Template Snippet (child.properties.j2)
11 ## child SSL config
12 {{ item }}.security.protocol=SSL
13 {{ item }}.ssl.truststore.location={{ kafka_secrets_dir }}/kafka
   .client.truststore.jks
14 {{ item }}.ssl.truststore.password={{ kafka_ssl_password }}
15 {{ item }}.ssl.keystore.location={{ kafka_secrets_dir }}/kafka.
   client.keystore.jks
16 {{ item }}.ssl.keystore.password={{ kafka_ssl_password }}
17 {{ item }}.ssl.key.password={{ kafka_ssl_password }}

```

Listing 5.9: Dynamic generation of Vault-injected MM2 configuration files (Root Node).

Because these highly sensitive `.properties` files are stored under the master hierarchy, they are fully protected by the host's `0700` POSIX boundary. A Malicious Observer navigating the host file system is categorically denied access to the MM2 configuration directory. Yet, operational continuity is preserved. When the N dedicated MM2 containers are initialized via Docker Compose, the Docker daemon utilizes its root-level privileges to bypass the traversal restriction. It securely mounts the `config/mirrormaker` directory as a read-only volume into the containers. The local MM2 process is then executed by passing its specific, isolated configuration file (e.g., `oda_child_1.properties`) as a command-line argument.

5.3.3 Replication Governance and Drift Control Implementation

The ultimate test of a Zero Trust architecture is its resilience against a Compromised Administrator, an adversary who has achieved root-level access on a deployed node. As mapped in the Threat Model, this attacker aims to sabotage the local pipeline, cause deliberate configuration drift, or leverage the compromised node to launch systemic attacks against the wider infrastructure (threats [T-CA.1] through [T-CA.4]).

To mitigate these catastrophic local failures and contain the systemic blast radius, the implementation combines application-level access controls (Kafka ACLs and Quotas)

with an autonomous infrastructure reconciliation loop driven by Ansible.

Child-Push Scenario

In the Child-Push architecture, replication governance is decentralized at the edge. If a Child node is compromised ([T-CA.1]), the adversary gains total control over the local MM2 engine. Without strict boundaries, the attacker could alter the MM2 configuration to inject fabricated data into unauthorized topics or flood the central Root broker with an overwhelming volume of requests.

To protect the core infrastructure, the Root node must enforce the principle of Least Privilege on all incoming client connections. Because Kafka metadata (including security rules) resides in-memory or in internal topics, these rules cannot be statically mounted via files (they must be injected into the broker at runtime). As defined in the orchestration lifecycle, the `start.yml` playbook waits for the Root's broker Java Virtual Machine (JVM) to initialize, and then dynamically delegates the `oda_root` role to inject the ACLs.

As shown in Listing 5.10, Ansible iterates over the `groups['child']` inventory to construct exact boundaries for each authorized Edge node. The Child (precisely the MM2 instance) is permitted to act as a producer on specific topics that are also prefixed with its own name (e.g., `oda_child_1.testingtopic`). To mitigate volumetric Endpoint Denial of Service, Network Quotas are simultaneously applied, capping the `producer_byte_rate` for each specific Client CN to 1 MB/s.

```

1 # 1. Allow child to write strictly to its designated business
   topics
2 - name: acl business topics
3   community.docker.docker_container_exec:
4     container: kafka_push
5     command: >
6       /kafka/bin/kafka-acls.sh --bootstrap-server localhost:{{
7         kafka_internal_port }}
8       --add
9       --allow-principal User:CN={{ item }}
10      --producer
11      --topic "{{ item }}.testingtopic"
12      --resource-pattern-type PREFIXED
13      loop: "{{ groups['child'] }}"
14
15 # 2. Enforce Bandwidth Quotas (1 MB/s limit per Child)
16 - name: quotas
17   community.docker.docker_container_exec:
18     container: kafka_push
19     command: >
20       /kafka/bin/kafka-configs.sh --bootstrap-server localhost
21       :{{ kafka_internal_port }}
22       --alter --add-config 'producer_byte_rate=1048576'
23       --entity-type users
24       --entity-name 'CN={{ item }}'
25       loop: "{{ groups['child'] }}"

```

Listing 5.10: Dynamic injection of Kafka ACLs and Quotas (Root Node).

While ACLs prevent data corruption on the Root, they do not prevent the local attacker from modifying the local `root.properties` file to cause a configuration drift. To achieve a best-effort autonomous Self-Healing (constrained by the polling interval window and the predefined scope of the managed configuration files), the Ansible `deploy.yml` playbook is scheduled via a cron job on the trusted Controller node to run continuously.

Leveraging Ansible's intrinsic idempotency, the templating engine forces (`force: yes`) the overwrite of any unauthorized alteration back to the baseline state declared. To apply the remediated configurations without breaking the infrastructure during regular provisioning, an *Event-Driven Handler Pipeline* is implemented.

As depicted in Listing 5.11, when the template detects a malicious drift, it triggers an event via the `notify` directive. The handlers listening to this event sequentially verify if the MM2 container is actively running and, only if true, issue a soft restart. This effectively flushes the poisoned configuration from the container's memory and loads the sanitized baseline, neutralizing the configuration drift.

```

1 # 1. Drift control enforcement (configure_mm2.yml)
2 - name: drift control to enforce mm2 config integrity
3   template:
4     src: ../../templates/root.properties.j2
5     dest: "{{ oda_deploy_path }}/config/mirrormaker/root.
6           properties"
7     force: yes
8   notify: restart mm2 container
9
10 # 2. Context-Aware Handlers (handlers/main.yml)
11 - name: check if mm2 container is currently running
12   command: docker ps -q -f name=mirrormaker_push
13   register: mm2_status
14   changed_when: false
15   listen: restart mm2 container
16
17 - name: restart mm2 container
18   command: docker restart mirrormaker_push
19   when: mm2_status.stdout != ""
20   listen: restart mm2 container

```

Listing 5.11: Event-Driven Handler Pipeline for Autonomous Drift Reconciliation (Child Node).

Root-Pull Scenario

In the Root-Pull architecture, the dynamic of systemic risk is completely inverted. The replication engine is centralized, meaning a compromised Child node ([T-CA.2]) is inherently quarantined: lacking outbound credentials and active pushing mechanisms, it cannot forcefully inject data into the core infrastructure. Conversely, a Compromised Administrator on the Root node ([T-CA.4]) represents the most critical threat, as the adversary could leverage the centralized MM2 engines to drain resources from all Edge nodes simultaneously or harvest unauthorized data streams.

To neutralize this threat, the Zero Trust boundaries must be enforced at the Edge. The distributed Child nodes, acting as passive servers, must restrict the central Root client strictly to its authorized operations.

During the Runtime Phase, the `start.yml` playbook delegates the application of security policies directly to the `oda_child` role. Since the Root node uses a distinct, granular certificate to connect to each Child, the Ansible playbook dynamically re-constructs the exact CN expected by extracting the numerical index from the Child's `inventory_hostname` (e.g., extracting 1 from `oda_child_1` to generate the principal `oda_root_1`).

As detailed in Listing 5.12, this dynamic principal is then used to inject the Kafka ACL. Symmetrically opposed to the Child-Push scenario, the Root is authorized using the `--consumer` macro, granting read-only access to the target topics. To prevent a compromised Root from executing a data-draining Denial of Service against the Edge, network Quotas are applied to cap the `consumer_byte_rate`.

```

1 # Extracting child index to compose the exact Root principal CN
2 - name: set root client CN fact
3   set_fact:
4     root_client_cn: "oda_root-{{ inventory_hostname.split('_') |
5       last }}"
6
7 # 1. Allow the specific Root client to consume business topics
8 - name: acl business topics
9   community.docker.docker_container_exec:
10     container: kafka_pull
11     command: >
12       /kafka/bin/kafka-acls.sh --bootstrap-server localhost:{{
13         kafka_internal_port }}
14       --add
15       --allow-principal User:CN={{ root_client_cn }}
16       --consumer
17       --topic "testingtopic"
18       --group "{{ root_client_cn }}-mm2"
19       --resource-pattern-type PREFIXED
20
21 # 2. Enforce Bandwidth Quotas against the Root (1 MB/s limit)
22 - name: quotas
23   community.docker.docker_container_exec:
24     container: kafka_pull
25     command: >
26       /kafka/bin/kafka-configs.sh --bootstrap-server localhost
27       :{{ kafka_internal_port }}
28       --alter --add-config 'consumer_byte_rate=1048576'
29       --entity-type users
30       --entity-name 'CN={{ root_client_cn }}'
```

Listing 5.12: Edge Defense: Dynamic extraction of the Root Principal and Consumer ACL/Quota injection (Child Node).

Simultaneously, the central Root node must be protected against configuration drift. If an attacker tampers with the centralized `config/mirrormaker` directory, they could hijack the replication pipelines of the entire distributed fleet. To maintain a best-effort autonomous Self-Healing, the `deploy.yml` playbook is scheduled continuously via `cron`.

However, managing the drift reconciliation for the Root-Pull topology introduces a

significant orchestration challenge: handling a 1-to-N scaling environment. The Root node hosts multiple independent MM2 containers. If only a single child's configuration file is maliciously altered, restarting the entire Docker Compose stack would cause an unacceptable pipeline outage for all the other healthy Edge nodes.

To solve this, the Event-Driven Handler Pipeline is engineered to be highly selective. When the templating task forces a state reconciliation on the `.properties` files within the `groups['child']` loop, it registers the execution result of each iteration (e.g., into a `mm2_template_result` variable) and emits a shared `restart mm2 containers` event. As demonstrated in Listing 5.13, the subscribed handlers leverage the `listen` directive to intercept this event. The first handler iterates through all Child nodes to register the runtime status of each corresponding MM2 container. The second handler then utilizes advanced loop controls (`loop_var` and `index_var`) to parse the resulting array (`mm2_status.results`). By cross-referencing the container's active status with the specific changed state of its configuration file at the matching index, it executes a targeted `docker restart` exclusively on the compromised instances.

```

1 # Context-Aware Handlers (handlers/main.yml)
2 - name: check if mm2 containers are currently running
3   command: docker ps -q -f name=mirrormaker_{{ item }}_pull
4   register: mm2_status
5   changed_when: false
6   loop: "{{ groups['child'] }}"
7   listen: restart mm2 containers
8
9 - name: execute the selective restart on running mm2 containers
10  command: docker restart mirrormaker_{{ my_container.item }}
11         _pull
12  loop: "{{ mm2_status.results }}"
13  loop_control:
14    loop_var: my_container
15    index_var: idx
16  when:
17    - my_container.stdout != ""
18    - mm2_template_result.results[idx].changed
19  listen: restart mm2 containers

```

Listing 5.13: Selective Event-Driven Handler Pipeline for 1-to-N Drift Reconciliation (Root Node).

This orchestration logic ensures that the Zero Trust architecture remains highly resilient. Compromised pipelines are autonomously sanitized and restarted without inflicting systemic downtime, demonstrating the operational effectiveness of a Security-as-Code implementation.

Chapter 6

Results Validation

Following the architectural design presented in Chapter 3, the formal Threat Modeling (Chapter 4), and the Security-as-Code implementation of Chapter 5, the final phase of this work requires an empirical evaluation of the deployed infrastructures. As dictated by the Threat Modeling Manifesto, the concluding question to address in any security lifecycle is: "Did we do a good job?".

To answer this question, this chapter presents a comprehensive security validation of the Hierarchical ODA ecosystem. The objective is to practically demonstrate that the theoretical countermeasures deployed via Ansible successfully mitigate the modeled adversarial profiles in a real-world scenario. This chapter first details the security testing methodology in Section 6.1, followed by the execution of targeted cyber-attacks to validate mTLS enforcement, secret management, and access controls in Section 6.2. Subsequently, it provides an experimental comparison between the Child-Push and Root-Pull topologies based on the operational and benchmarking insights gathered during the deployments in Section 6.3 and 6.4.

6.1 Validation Methodology

To ensure a rigorous evaluation, the testing methodology adopted in this chapter relies on simulating real-world attack vectors against the fully provisioned infrastructure. Rather than relying solely on theoretical assertions, the resilience of the Security-as-Code templates is assessed by actively attempting to circumvent the deployed security boundaries.

The validation is structured around the three adversarial profiles defined in the Threat Model: the Network Adversary, the Malicious Observer, and the Compromised Administrator. For each profile, targeted operations are executed to verify the correct enforcement of several key security perimeters. First, the evaluation assesses transport security by confirming that mutual TLS effectively rejects unauthenticated or misconfigured network

connections. Subsequently, file system hardening is tested to ensure that POSIX permissions prevent local privilege escalation and information disclosure. The tests also verify application-level authorization, ensuring that Kafka ACLs uphold the least privilege principle to contain lateral movement and data poisoning attempts. Finally, state reconciliation is evaluated by observing whether the Ansible idempotency engine can autonomously detect and remediate configuration drifts induced by a simulated root-level compromise.

All tests were conducted on the live VMs provisioned on the University of Pisa infrastructure, ensuring that the results accurately reflect the behavior of the system in a realistic deployment environment. To provide a comprehensive evaluation, the testing suite was executed sequentially on both the Child-Push and Root-Pull architectural topologies.

6.2 Threat Mitigation Testing

Sections 6.2.1, 6.2.2, and 6.2.3 detail the execution and the collected empirical results of the security validation phase. For clarity, the results are presented chronologically, escalating the privileges and capabilities of the simulated attacker from an external network position up to full administrative access.

6.2.1 Mitigating the Network Adversary

The Network Adversary operates within the untrusted transport network (the University VPN). As defined by the Threat Model, this profile attempts to sniff traffic, spoof identities, or establish unauthorized connections to the exposed endpoints. To mitigate these threats, the infrastructure enforces mTLS. The tests validate that the endpoints actively reject any connection lacking a valid, internal CA-signed client certificate.

Child-Push Scenario

In the Child-Push architecture, the central Root node exposes its Kafka broker (port 9094) to accept incoming replication streams from the peripheral Child nodes. To simulate an attacker attempting to establish a secure tunnel without the proper cryptographic identity, the `openssl s_client` utility was executed from an unauthorized machine on the VPN (the Ansible Controller machine).

As demonstrated in Listing 6.1, the Root broker successfully presents its server certificate, but the TLS handshake is abruptly terminated. The OpenSSL output explicitly shows an SSL alert number 116 (Certificate Required). Because the `ssl.client.auth=required`

parameter is enforced by Ansible, the broker proactively drops the connection when the client fails to provide a valid certificate.

```

1 $ openssl s_client -connect XXX.XXX.XXX.XXX:9094
2 Connecting to XXX.XXX.XXX.XXX
3 CONNECTED(00000003)
4 [...]
5 ---
6 Acceptable client certificate CA names
7 CN=ODA-Internal-CA
8 [...]
9 Verify return code: 19 (self-signed certificate in certificate
   chain)
10 ---
11 40F7D4B913710000:error:0A00045C:SSL routines:ssl3_read_bytes:
    tlsv13 alert certificate required:ssl/record/rec_layer_s3.c
    :914:SSL alert number 116

```

Listing 6.1: Client-side OpenSSL output showing the TLS handshake rejection by the Root broker.

To verify that the Network Adversary cannot bypass the TLS layer to extract Kafka metadata (such as topic names or cluster configurations), the standard `kcat` utility was used to query the exposed broker.

As shown in Listing 6.2, the client is immediately disconnected before any Kafka-specific API request can be completed.

```

1 $ kcat -b XXX.XXX.XXX.XXX:9094 -L
2 %6|1774349167.980|FAIL|rdkafka#producer-1| [thrd:XXX.XXX.XXX.XXX
   :9094/bootstrap]: XXX.XXX.XXX.XXX:9094/bootstrap:
   Disconnected while requesting ApiVersion: might be caused by
   incorrect security.protocol configuration (connecting to a
   SSL listener?)
3 % ERROR: Failed to acquire metadata: Local: Broker transport
   failure

```

Listing 6.2: Client-side `kcat` output demonstrating the inability to fetch Kafka data/metadata.

Simultaneously, the continuous monitoring of the Root node's internal logs confirms that the infrastructure successfully detected and blocked the intrusion attempt at the network socket layer, as captured in Listing 6.3.

```
[2026-03-24 10:46:07,408] INFO [SocketServer listenerType=
  ZK_BROKER, nodeId=0] Failed authentication with /XXX.XXX.XXX.
  XXX (channelId=XXX.XXX.XXX.XXX:9094-XXX.XXX.XXX.XXX:23440-0)
  (SSL handshake failed) (org.apache.kafka.common.network.
  Selector)
```

Listing 6.3: Root broker internal logs confirming the dropped connections.

These results indicate that the Network Adversary is effectively mitigated. Lacking the appropriate CA-signed materials, the attacker is cryptographically isolated from the application layer, ensuring data confidentiality and system integrity in the Child-Push topology.

Root-Pull Scenario

In the inverted Root-Pull architecture, the attack surface shifts: the peripheral Edge nodes expose their local Kafka brokers (port 9095) to allow the central Cloud to pull the replicated data. The Network Adversary, therefore, targets the factory environment directly rather than the central core.

To validate the transport security at the Edge, the exact same unauthenticated probing methodology was executed against a Child node (oda_child_1). As demonstrated in Listing 6.4, the symmetric application of the Security-as-Code templates guarantees identical resilience. The Child's broker successfully presents its server certificate (CN=oda_child_1) but instantly drops the connection with an SSL alert number 116 when the attacker fails to provide a valid mTLS client certificate. The subsequent `kcat` application-layer probe is similarly rejected at the transport level.

```

1 $ openssl s_client -connect XXX.XXX.XXX.XXX:9095
2 Connecting to XXX.XXX.XXX.XXX
3 CONNECTED(00000003)
4 [...]
5 subject=CN=oda_child_1
6 issuer=CN=ODA-Internal-CA
7 [...]
8 Verify return code: 19 (self-signed certificate in certificate
   chain)
9 ---
10 40D78BAB2D7B0000:error:0A00045C:SSL routines:ssl3_read_bytes:
    tlsv13 alert certificate required:ssl/record/rec_layer_s3.c
    :914:SSL alert number 116
11
12 $ kcat -b XXX.XXX.XXX.XXX:9095 -L
13 % ERROR: Failed to acquire metadata: Local: Broker transport
    failure

```

Listing 6.4: Symmetric TLS handshake rejection executed by the exposed Child broker in the Root-Pull topology.

This outcome confirms that the Zero Trust perimeter is enforced regardless of the topological direction. The Edge network remains cryptographically sealed against external Network Adversaries, restricting unauthorized reconnaissance or data interception.

6.2.2 Mitigating the Malicious Observer

The Malicious Observer represents an adversary who has successfully bypassed the external perimeter and gained unprivileged shell access to a target node. As modeled, this attacker attempts to perform local reconnaissance to harvest cryptographic materials (mTLS certificates) or plain-text credentials stored within configuration files. To mitigate this Information Disclosure threat, the Ansible playbooks enforce a strict 0700 POSIX permission boundary on the base deployment directories.

Child-Push Scenario

In the Child-Push architecture, the Edge node stores the Client Keystores and the `.properties` file required by MM2 to authenticate and push data to the central Root. If compromised, these assets could allow the attacker to pivot and attack the Cloud core.

To validate the file system hardening, a local privilege escalation attempt was simulated using the `nobody` account, which represents the lowest-privileged standard user on a Linux system. The attacker first attempts to list the contents of the base deployment directory to map the file structure. Subsequently, they attempt to directly read the binary `.jks`

keystore and the MM2 configuration file containing the decrypted Vault secrets. As demonstrated in Listing 6.5, these attempts are rejected by the Operating System with a Permission denied error.

```
1 # 1. Attacker attempts to list the base directory contents
2 $ sudo -u nobody ls -la <oda_deploy_path>
3 ls: cannot open directory '<oda_deploy_path>': Permission denied
4
5 # 2. Attacker attempts to read the mTLS Client Keystore
6 $ sudo -u nobody cat <oda_deploy_path>/secrets/kafka.client.
   keystore.jks
7 cat: <oda_deploy_path>/secrets/kafka.client.keystore.jks:
   Permission denied
8
9 # 3. Attacker attempts to read the Vault-injected MM2
   configuration
10 $ sudo -u nobody cat <oda_deploy_path>/config/mirrormaker/root.
   properties
11 cat: <oda_deploy_path>/config/mirrormaker/root.properties:
   Permission denied
```

Listing 6.5: Terminal output simulating a Malicious Observer attempting to access restricted deployment artifacts.

Because the base directory is strictly owned by the deployment user with 0700 permissions, the Linux kernel fundamentally blocks any unprivileged traversal. The attacker cannot list the directory contents, nor can they access the nested `secrets` or `config` subdirectories, even if they correctly guess the absolute paths.

This identical test yields the exact same results when executed on the central Root node, ensuring that the Server Keystores and administrative credentials stored on the core Cloud infrastructure remain isolated from an unauthorized local observer.

Root-Pull Scenario

In the Root-Pull architecture, the central Root node becomes an high-value target for a Malicious Observer. Because it hosts the individual MM2 instances for every connected Edge node, its file system aggregates the cryptographic materials and Vault-injected configurations for the entire ecosystem.

To validate the protection of these aggregated secrets, the local privilege escalation test was repeated on the central Cloud VM. As shown in Listing 6.6, an unprivileged user attempting to traverse the deployment directory to extract a specific tenant's configuration (e.g., `oda_child_1`) is immediately blocked by the Operating System.

```
1 # 1. Attacker attempts to list a specific Child's secrets
   directory
2 $ sudo -u nobody ls -la <oda_deploy_path>/secrets/oda_child_1
3 ls: cannot access '<oda_deploy_path>/secrets/oda_child_1':
   Permission denied
4
5 # 2. Attacker attempts to read the Vault-injected MM2
   configuration for the Child
6 $ sudo -u nobody cat <oda_deploy_path>/config/mirrormaker/
   oda_child_1.properties
7 cat: <oda_deploy_path>/config/mirrormaker/oda_child_1.properties
   : Permission denied
```

Listing 6.6: Terminal output demonstrating OS-level rejection of unauthorized access to aggregated Edge secrets on the Root node.

The symmetric enforcement of the 0700 POSIX boundary by the Ansible playbooks ensures that even on a shared, multi-tenant central node, the infrastructure secrets remain cryptographically isolated.

6.2.3 Mitigating the Compromised Administrator

The Compromised Administrator represents the most severe adversarial profile. In this scenario, the attacker has achieved root-level execution privileges on a target node. The strict POSIX file system defenses are completely bypassed, granting the attacker full access to the cryptographic materials and configuration files. As defined in the Threat Model, the attacker's objective is to hijack the local replication pipeline to sabotage the wider infrastructure (e.g., injecting unauthorized data, causing configuration drift).

To evaluate the defense-in-depth strategy against this profile, the assessment verifies both the centralized application-level access controls (Kafka ACLs) and the autonomous Infrastructure-as-Code reconciliation loop.

Child-Push Scenario

In the Child-Push architecture, the Compromised Administrator controls the peripheral Edge node. By hijacking the active MM2 instance, the attacker attempts to weaponize the legitimate client certificate to poison the central Root cluster.

Test 1: Replication Pipeline Hijacking (Data Poisoning) To simulate the attack, the adversary first creates a rogue local topic named `malicioustopic` using the Kafka CLI tools. Utilizing their root privileges, they manually modify the `root.properties` MM2

configuration file stored on the host. The replication directive is maliciously altered from the restricted baseline (`topics=testingtopic.*`) to a wildcard regular expression (`topics=.*`) to force the synchronization of all local topics, including the rogue one, toward the Cloud core. The attacker then manually restarts the `mirrormaker_push` container to apply the poisoned configuration.

As the compromised MM2 instance connects to the Root and attempts to create and produce data into `oda_child_1.malicioustopic`, the central broker evaluates the request against the dynamically provisioned ACLs. As demonstrated by the MM2 worker logs in Listing 6.7, the attack is blocked.

```

1 [2026-03-24 13:59:58,110] WARN [MirrorSourceConnector|worker]
   Could not create topic oda_child_1.malicioustopic. (org.
   apache.kafka.connect.mirror.MirrorSourceConnector:501)
2 java.util.concurrent.CompletionException: org.apache.kafka.
   common.errors.TopicAuthorizationException: Authorization
   failed.
3 [...]
4 Caused by: org.apache.kafka.common.errors.
   TopicAuthorizationException: Authorization failed.
```

Listing 6.7: MM2 logs on the compromised Child node showing the ACL rejection from the Root broker.

The Root node's `--producer` ACL strict prefix mapping (`oda_child_1.testingtopic`) effectively prevents the compromised client from writing outside its designated boundary, successfully containing the blast radius at the application layer.

Test 2: Tenant Impersonation (Lateral Movement) To escalate the attack, the Compromised Administrator attempts a Lateral Movement maneuver by impersonating a different tenant within the ecosystem. The goal is to perform a Data Poisoning attack by injecting fabricated data into the replication pipeline of another Edge node (e.g., `oda_child_2`).

The adversary manipulates the local `root.properties` file, replacing all references of `oda_child_1` with `oda_child_2`. This alters the MM2 internal behavior, forcing it to claim the identity of the second Child and attempt to write into its designated Cloud topics (e.g., `oda_child_2.testingtopic` and its internal offset topics).

As shown in Listing 6.8, when the tampered MM2 container is restarted, the replication process encounters a `TopicAuthorizationException` during its startup phase.

```
1 [2026-03-24 14:29:49,555] INFO [Consumer clientId=oda_child_2-  
mm2-oda_child_2->root-offsets, groupId=oda_child_2-mm2]  
Cluster ID: YqIaiUiAQ62WnFj7VnQsFA  
2 [2026-03-24 14:29:49,559] ERROR [Worker clientId=oda_child_2->  
root, groupId=oda_child_2-mm2] Uncaught exception in herder  
work thread, exiting:  
3 org.apache.kafka.common.errors.TopicAuthorizationException: Not  
authorized to access topics: [mm2-offsets-oda_child_2]  
4 [2026-03-24 14:29:49,567] INFO Kafka MirrorMaker stopping
```

Listing 6.8: Fatal exception in MM2 logs blocking the tenant impersonation attack.

While the attacker can freely manipulate the local application-level configuration to claim a false identity (`oda_child_2`), the underlying TLS tunnel remains cryptographically bound to the node's physical certificate (`CN=oda_child_1`). The central Root broker evaluates the ACLs based exclusively on this verified TLS Principal, detecting the discrepancy and dropping the unauthorized operations.

Test 3: Autonomous Configuration Reconciliation (Self-Healing) While the Root node is safe from data poisoning, the Child node remains in a state of Configuration Drift due to the locally tampered `root.properties` file. To validate the Autonomous Drift Control engineered in Section 5.3.3, the declarative `deploy.yml` playbook was executed, simulating its automated cron scheduling.

As captured in Listing 6.9, Ansible's idempotency engine scans the target's current state and instantly detects the malicious modification on the templated file. It autonomously overwrites the poisoned file with the trusted baseline, reporting a `changed` status. This state alteration correctly triggers the Event-Driven Handler Pipeline, which evaluates the container's runtime status and executes a soft restart on the active `mirrormaker_push` container to flush the malicious configuration from memory.

```
1 TASK [oda_child : drift control to enforce mm2 config integrity]
2 ok: [oda_child_2]
3 changed: [oda_child_1]
4
5 RUNNING HANDLER [oda_child : check if mm2 container is currently
   running]
6 ok: [oda_child_1]
7
8 RUNNING HANDLER [oda_child : restart mm2 container]
9 changed: [oda_child_1]
10
11 PLAY RECAP
12 oda_child_1 : ok=11    changed=2    unreachable=0
   failed=0
```

Listing 6.9: Ansible output demonstrating autonomous drift detection and targeted remediation.

Following the playbook execution, a verification of the local file system confirmed that the `topics` directive was successfully reverted to `testingtopic.*`, and the Docker engine verified that the MM2 container was freshly restarted without any human intervention. This test illustrates that the Security-as-Code implementation provides a highly resilient, self-healing infrastructure capable of autonomously recovering from severe local administrative compromises.

Test 4: Endpoint Denial of Service Prevention (Network Quotas) A Compromised Administrator could leverage the hijacked node to launch an Endpoint DoS attack, flooding the Cloud core with a massive volume of illegitimate replication traffic to exhaust centralized resources.

To mitigate this volumetric threat, the Root broker dynamically enforces Network Quotas based on the verified TLS Principal. Specifically, a `producer_byte_rate` limit of 1 MB/s (1048576 bytes) is applied to `CN=oda_child_1`. To evaluate the efficacy of this boundary, the `kafka-producer-perf-test.sh` benchmarking tool was executed directly from within the compromised MM2 container, effectively simulating a weaponized workload utilizing the legitimate mTLS client certificate.

As captured in Listing 6.10, the broker successfully mitigates the flooding attempt through TCP throttling.

```

1 $ /kafka/bin/kafka-producer-perf-test.sh --topic oda_child_1.
   testingtopic1 --num-records 50000 --record-size 10240 --
   throughput -1 --producer.config quota-test.properties --
   producer-props bootstrap.servers=XXX.XXX.XXX.XXX:9094
2
3 1114 records sent, 220.9 records/sec (2.16 MB/sec), 872.5 ms avg
   latency, 3987.0 ms max latency.
4 5 records sent, 1.0 records/sec (0.01 MB/sec), 7018.6 ms avg
   latency, 9039.0 ms max latency.
5 1107 records sent, 185.6 records/sec (1.81 MB/sec), 11498.4 ms
   avg latency, 14605.0 ms max latency.
6 5 records sent, 1.0 records/sec (0.01 MB/sec), 17632.0 ms avg
   latency, 19648.0 ms max latency.
7 1107 records sent, 185.8 records/sec (1.81 MB/sec), 20264.7 ms
   avg latency, 21985.0 ms max latency.

```

Listing 6.10: Performance test output demonstrating Kafka’s native quota throttling and induced latency penalties.

The output illustrates the mechanics of Kafka’s window-based quota enforcement. The broker permits a brief initial burst, immediately detects the quota violation across its evaluation window, and penalizes the client by artificially inflating the response latency (reaching over 21,000 ms). This delay forces the attacker’s client into an extended backoff state (momentarily dropping the throughput to 0.01 MB/sec), effectively capping the overall moving average to the defined 1 MB/s boundary and preserving the Core’s availability.

Theoretical Analysis: Root Node Compromise and Blast Radius Containment For the sake of completeness, the assessment must also consider the symmetrical threat: a Compromised Administrator gaining root access to the central Root node. In this scenario, the attacker gains total control over the central Kafka broker, the Server Keystores, and the aggregated data. While this represents a critical compromise for the Cloud core, the structural design of the Child-Push topology inherently contains the blast radius.

Because the data flow is unidirectional (Edge-to-Cloud) and the Edge nodes act exclusively as active clients, the peripheral Child instances do not expose any inbound listeners to the transport network. Furthermore, the Root node does not possess any cryptographic client identities capable of authenticating against the Edge. Consequently, it is architecturally prevented for the compromised Cloud attacker to pivot back into the local Edge networks to extract local data or execute reverse-shell maneuvers.

Root-Pull Scenario

In the Root-Pull architecture, the paradigm is inverted: the central Cloud (Root) actively connects to the peripheral Edge networks. Consequently, if the central Root node is compromised, the primary threat is a top-down attack where the adversary attempts to abuse the centralized replication pipeline to either exfiltrate private factory data or push malicious commands down to the Edge.

To validate the Zero Trust boundary from the Cloud to the Edge, the assessment verifies that the Edge nodes mathematically restrict the Root's capabilities exclusively to narrow, authorized read operations.

Test 1: Data Exfiltration Prevention (Unauthorized Consume) The Compromised Administrator on the Cloud attempts to exfiltrate sensitive local data from the Edge. To simulate this, a local-only topic (`privatedata`) is created on the Child broker. The attacker then manipulates the Vault-injected `oda_child_1.properties` file on the Root node, altering the replication directive to a wildcard (`topics=.*`) to force MM2 to pull all local factory topics. The MM2 container is subsequently restarted.

As MM2 connects to the Child broker, the Edge's ACLs evaluate the incoming requests. Because the `CN=oda_root` Principal lacks the `Describe` permission for the `privatedata` topic, the Child broker securely masks its existence, silently omitting it from the metadata response and preventing the replication task from even discovering the sensitive asset.

This demonstrates that the Cloud core is strictly confined to a Least Privilege boundary, guaranteeing that internal industrial data remain secure.

Test 2: Cloud-to-Edge Data Poisoning (Unauthorized Produce) Having failed to silently exfiltrate private data, the Compromised Administrator attempts an active Data Poisoning attack. The objective is to push fabricated data or destructive commands down to the factory floor by writing into the Edge's local topics.

To execute this top-down lateral movement, the attacker leverages the interactive shell of the compromised MM2 container on the Root node, which inherently possesses the valid `CN=oda_root_1` client certificate. Using the native Kafka console producer utility, the attacker formats a malicious payload and targets the exposed Child broker.

As demonstrated in Listing 6.11, the intrusion attempt is rejected by the Edge broker with a `ClusterAuthorizationException` before the payload can even be processed.

```
1 $ echo "malicious command from cloud" | /kafka/bin/kafka-console
  -producer.sh --bootstrap-server XXX.XXX.XXX.XXX:9095 --topic
  testingtopic1 --producer.config test.properties
2
3 >>[2026-03-24 17:27:39,947] ERROR [Producer clientId=console-
  producer] Aborting producer batches due to fatal error (org.
  apache.kafka.clients.producer.internals.Sender)
4 org.apache.kafka.common.errors.ClusterAuthorizationException:
  Cluster authorization failed.
5 [2026-03-24 17:27:39,950] ERROR Error when sending message to
  topic testingtopic1 with key: null, value: 28 bytes with
  error:
6 org.apache.kafka.common.errors.ClusterAuthorizationException:
  Cluster authorization failed.
```

Listing 6.11: Terminal output showing the Edge broker rejecting an unauthorized write attempt from the compromised Cloud.

This result confirms the efficacy of the Least Privilege principle enforced via Ansible. The Child's ACLs grant the Cloud identity only `--consumer` (Read and Describe) privileges, denying any write operation.

Test 3: Selective Autonomous Configuration Reconciliation To counter the threat of configuration drift on the central Cloud, the Root-Pull implementation of the IaC must handle 1-to-N relationships. Because the Root node hosts multiple MM2 instances, the autonomous healing process must be capable of surgically remediating only the compromised components.

To validate this Selective Self-Healing, the `oda_child_1.properties` file, previously tampered with during the exfiltration test, was left in its poisoned state, while the configuration for `oda_child_2` was verified to be compliant. The declarative Ansible deployment playbook was then executed to simulate a scheduled reconciliation cycle.

As shown in Listing 6.12, the Ansible idempotency engine iterates through the tenant configurations via dynamic loops. It identifies the drift exclusively on the first child's file (`changed: [oda_root] => (item=oda_child_1)`), whilst evaluating the second tenant's file as fully compliant (`ok: [oda_root] => (item=oda_child_2)`).

```

1 TASK [oda_root : drift control to enforce mm2 config integrity]
2 changed: [oda_root] => (item=oda_child_1)
3 ok: [oda_root] => (item=oda_child_2)
4
5 RUNNING HANDLER [oda_root : check if mm2 containers are
   currently running]
6 ok: [oda_root] => (item=oda_child_1)
7 ok: [oda_root] => (item=oda_child_2)
8
9 RUNNING HANDLER [oda_root : execute the restart on running mm2
   containers]
10 changed: [oda_root] => (item=oda_child_1)
11 skipping: [oda_root] => (item=oda_child_2)

```

Listing 6.12: Ansible output demonstrating surgical drift detection and selective container restart on a multi-tenant Cloud node.

The state alteration triggers an advanced event-driven handler pipeline equipped with index-aware loop controls. The handler selectively executes the soft restart exclusively on the affected container, actively bypassing the healthy one (**skipping**). This demonstrates that the Security-as-Code approach can manage complex, multi-tenant Cloud topologies, dynamically healing localized compromises while maintaining stability for the rest of the infrastructure.

Theoretical Analysis: Child Node Compromise and Volumetric Mitigation For completeness, the inverse scenario must be evaluated: a Compromised Administrator gaining root access to an Edge node in the Root-Pull topology. In this state, the attacker can manipulate local Edge data, which will inevitably be pulled by the Cloud. However, the architectural design strictly contains the blast radius to the local tenant.

Because the Edge node acts exclusively as a passive server and possesses no client credentials to authenticate against the Cloud core, it is topologically restricted from initiating outbound connections to pivot into the central Root.

Furthermore, the Root-Pull paradigm inherently mitigates Endpoint DoS attacks. Unlike the Push model, where a compromised Edge can actively flood the Cloud, the Pull model shifts the flow control to the central consumer. If an attacker generates massive volumes of local data to saturate the pipeline, the Cloud's MM2 instance dictates the fetching pace. To symmetrically protect the Edge network's bandwidth from being overwhelmed by a compromised or misconfigured Cloud consumer, `fetch_byte_rate` Network Quotas can be applied on the Child broker, providing similar volumetric containment.

6.3 Replication Performance Benchmarking

While Sections 6.2.1, 6.2.2, and 6.2.3 validated the logical resilience of the security measures implemented, adopting a Security-as-Code architecture over a distributed network introduces operational overhead. This section aims to quantify the physical impact of mTLS encapsulation and MM2 orchestration on the data pipeline.

To ensure an effective empirical evaluation, the benchmark utilized native Apache Kafka tools (such as `kafka-producer-perf-test.sh`) to inject a synthetic workload of 500,000 records, each 1 KB in size. To isolate the cryptographic and computational limits of the pipeline, the 1 MB/s network quotas, previously validated in Section 6.2.3, were dynamically overridden to 100 MB/s prior to the execution of the benchmarking tests.

6.3.1 Child-Push Evaluation

In the Child-Push architecture, the Edge node actively executes the MM2 engine, distributing the computational burden of data extraction, encryption, and routing to the periphery.

During the load injection, the local Edge broker successfully ingested data at a sustained rate of 23.92 MB/s. However, the End-to-End (E2E) replication throughput, actively routed through the mTLS-encrypted connection to the Cloud core, stabilized at approximately 1.93 MB/s. To contextualize this discrepancy, a baseline network evaluation via `iperf3` revealed an available raw TCP bandwidth of ~900 MB/s between the testbed VMs. This suggests that the bottleneck is not the transport network. Instead, the continuous mTLS encapsulation of high-frequency, small-payload batches heavily constrains the pipeline's maximum speed.

Resource monitoring, sampled at 1Hz frequency via Docker daemon statistics, demonstrated that the Edge container easily sustains this workload. While MM2 experienced an initial start-up burst (peaking at 86.28% CPU) during tunnel negotiation and buffer allocation, it quickly settled into a stable regime of 4-8% CPU utilization. The memory footprint grew steadily but safely plateaued at ~379 MiB. This indicates that the Child-Push topology can be safely deployed on resource-constrained Edge gateways without inducing local hardware exhaustion.

6.3.2 Root-Pull Evaluation

The Root-Pull architecture inverts the replication flow, centralizing the MM2 engine on the Root node, which actively polls the exposed Edge broker to pull the local data.

Under identical stress conditions, the Edge node ingested local telemetry at 25.85 MB/s. The observed E2E replication throughput on the Cloud side was 1.84 MB/s. This

value is statistically equivalent to the Child-Push results, demonstrating that reversing the direction of the Kafka replication protocol does not bypass the fundamental cryptographic and application-level bottlenecks.

The critical distinction lies in the architectural resource allocation. The centrally hosted MM2 container registered a slightly lower start-up burst (66.07% CPU) and maintained a stable regime of 4-7% CPU utilization. The memory footprint peaked at ~332 MiB. While these metrics indicate a highly efficient process, they represent a centralized operational cost per child.

6.3.3 Benchmarking Summary

Table 6.1 summarizes the empirical data collected. The metrics clearly illustrate that while E2E network throughput is capped by the secure protocol overhead at nearly 2 MB/s regardless of the chosen topology, the placement of the MM2 engine dictates a shift in infrastructural scalability.

Metric	Child-Push Topology	Root-Pull Topology
Payload Size	1 KB	1 KB
Total Messages	500,000	500,000
Local Edge Ingestion	23.92 MB/s	25.85 MB/s
E2E Replication Throughput	~1.93 MB/s	~1.84 MB/s
MM2 Peak CPU (Start-up)	86.28%	66.07%
MM2 Avg CPU (Regime)	~4–8%	~4–7%
MM2 Max RAM Usage	~379 MiB	~332 MiB

Table 6.1: Replication performance comparison between the Child-Push and Root-Pull topologies under a 500,000-record workload.

6.4 Experimental Comparison: Child-Push vs. Root-Pull

The empirical validation conducted in this chapter confirms that both the Child-Push and Root-Pull architectures can be successfully secured against the modeled adversarial profiles. Through the application of IaC, mutual TLS, and granular access controls, the Zero Trust boundaries were upheld in both paradigms. However, the operational execution of these deployments highlighted profound differences in deployability, orchestration complexity, and trust dynamics, requiring a comparative analysis to guide real-world adoption.

6.4.1 Network Deployability and the VPN Context

From a networking perspective, the experimental deployment validated the theoretical advantage of the Child-Push topology. Because the peripheral nodes act as active clients, their outbound replication traffic natively traverses Network Address Translation (NAT) boundaries and local firewalls. In a scenario involving highly distributed or mobile IoT devices operating over public WANs, the Child-Push model is logistically superior, requiring only a single, stable public endpoint at the Cloud core.

Conversely, the Root-Pull architecture proved to be highly dependent on the underlying transport network. Because the central Root node must actively initiate connections to the Edge, it requires the peripheral brokers to be directly addressable. In the testbed environment, this was seamlessly achieved due to the overarching University VPN, which provided a flat, routable overlay network. However, deploying a Root-Pull topology over a standard public internet connection without a dedicated VPN would pose significant operational challenges, limiting its applicability in environments lacking advanced network orchestration.

6.4.2 Orchestration Complexity and Secret Management

The implementation of Security-as-Code revealed a stark contrast in the orchestration burden required to maintain the two topologies.

The Child-Push architecture benefits from a decentralized and uniform deployment logic. The Ansible roles executed at the Edge are relatively straightforward (a 1-to-1 mapping), and the core Cloud node simply acts as a passive receiver. The cryptographic identity management is strictly compartmentalized, minimizing the risk of systemic credential exposure.

In contrast, the Root-Pull topology centralized the replication governance, which drastically increased the IaC complexity. As demonstrated during the configuration drift tests, managing a 1-to-N scaling environment on a single Root node required advanced Ansible logic (e.g., incorporating dynamic loop controls, index-aware condition parsing, and highly selective event handlers) to avoid inducing systemic downtime during local remediations. Furthermore, this centralization transforms the Root node into a high-value target: the aggregation of all Edge cryptographic materials and MM2 configurations on a single machine inherently increases the systemic risk if the core is breached.

6.4.3 Trust Assumptions and Kafka Best Practices

A significant comparative insight emerges when evaluating the architectures against varying trust models. The strict Zero Trust assumption applied throughout this thesis, treating

both the Edge and the Cloud as potentially compromised, naturally favors the Child-Push topology. By preventing any inbound connections to the Edge, the Child-Push model logically air-gaps the Operational Technology network from top-down lateral movements, effectively isolating central Cloud breaches.

However, in enterprise environments where the Zero Trust assumption can be safely relaxed for the core infrastructure, treating the central Cloud as highly secure, the Root-Pull architecture presents notable advantages. Centralizing the MM2 instances aligns with the official Kafka best practice to "consume from remote, produce to local". This approach shifts the computational load and memory buffering associated with data replication away from resource-constrained Edge nodes and onto the scalable Cloud infrastructure. If the Root node is considered fully trusted, the Root-Pull model offers strong centralized governance, allowing administrators to quarantine rogue Edge nodes instantly without relying on complex, edge-side dynamic ACLs.

6.4.4 Architectural Recommendations

Ultimately, there is no single optimal solution for Hierarchical ODA deployments: the choice is dictated by the specific industrial context.

The **Child-Push architecture** is recommended for highly decentralized, Zero Trust environments operating over untrusted networks. It prioritizes the security and isolation of the Edge at the cost of higher peripheral resource consumption and complex core-side authorization management.

The **Root-Pull architecture**, conversely, is a highly suitable choice for tightly integrated environments (e.g., enterprise VPNs) where the central infrastructure is trusted. It offloads the computational overhead of data replication from the Edge, optimizes data ingestion according to Kafka best practices, and provides strong centralized governance over the entire replication pipeline.

To guide a real-world architectural selection, Figure 6.1 proposes a sequential decision tree. The framework synthesizes the comparative analysis into three cascading nodes: network deployability (a hard logistical constraint), core trust assumptions (a policy constraint), and governance prioritization (an operational constraint).

The decision-making process begins by evaluating the physical network topology. Drawing from the deployability analysis detailed in Section 6.4.1, if the Edge nodes are not directly addressable by the central Cloud, the Root-Pull topology becomes logistically unfeasible. In such scenarios, the Child-Push architecture is the mandatory choice.

If the underlying transport network renders the Edge nodes directly addressable (e.g., through a dedicated VPN overlay), the framework shifts to evaluate the required security posture. Following the trust assumptions discussed in 6.4.3, if a strict Zero Trust perimeter

must be maintained against the Cloud core, the Child-Push architecture should be selected to isolate the Edge environment.

Conversely, if the central Root is safely considered a trusted environment, the architectural decision relies entirely on operational preferences. As evaluated in Section 6.4.2 and 6.4.3, organizations prioritizing centralized governance and aiming to offload the computational overhead of data replication from the resource-constrained Edge will find the Root-Pull topology highly suitable. On the other hand, if a decentralized and uniform deployment logic is preferred, the Child-Push paradigm remains the best alternative.

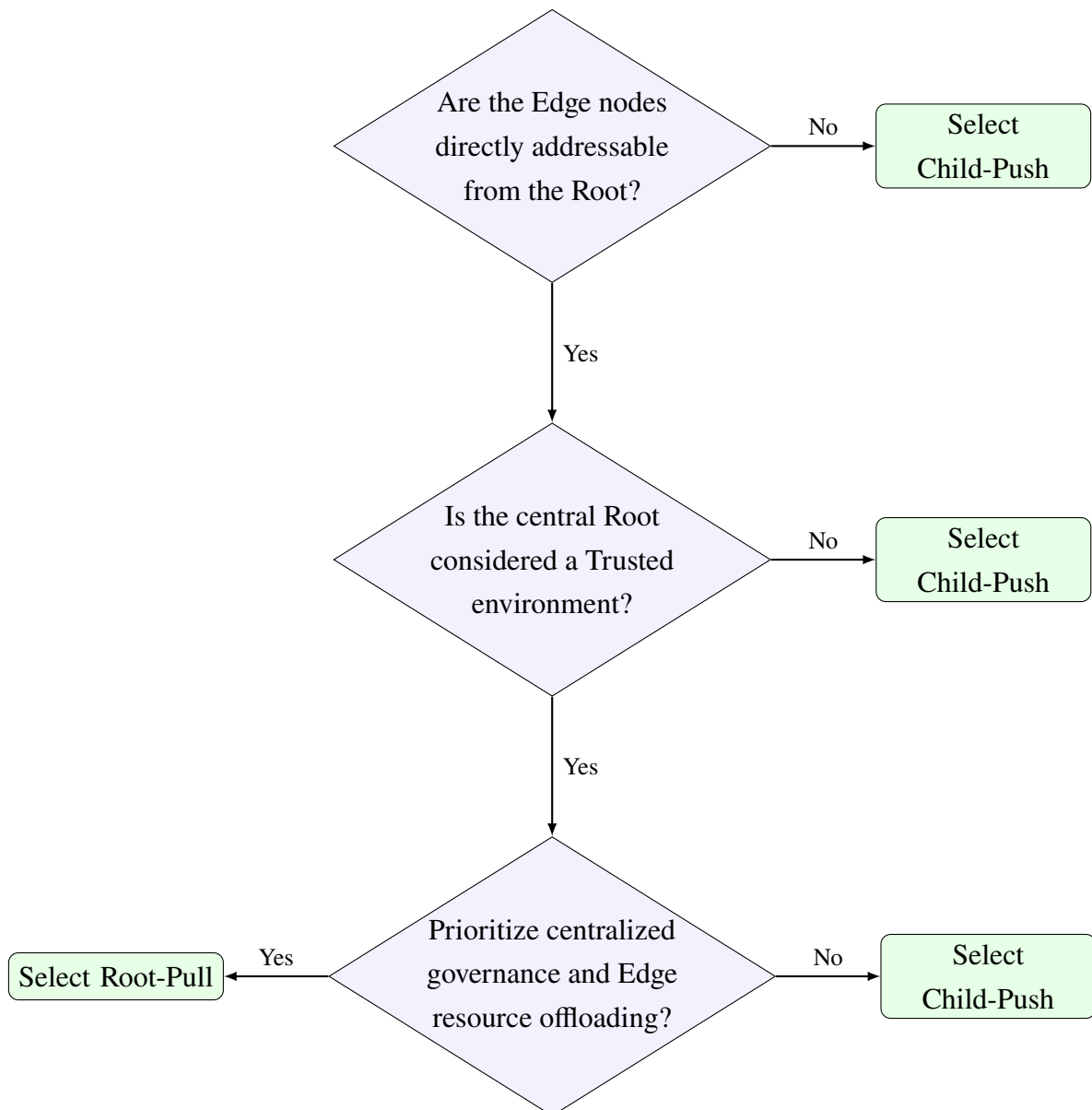


Figure 6.1: Decision tree framework for selecting the optimal Hierarchical ODA topology.

Chapter 7

Conclusions

The exponential growth of Industry 4.0 and IoT ecosystems necessitates the transition of data streaming platforms from isolated, standalone environments into geographically distributed Edge-to-Cloud architectures. However, as demonstrated throughout this thesis, expanding a microservice-based application like ODA across untrusted transport networks introduces significant security vulnerabilities. This work aimed to address these challenges by bridging the operational need for hierarchical scalability with the rigorous enforcement of a Zero Trust security paradigm.

Operating under the assumption that the baseline application must be treated as a trusted "black box", this research delegated the security enforcement primarily to the infrastructure and transport layers. By adopting a Security-as-Code approach, the thesis demonstrated that declarative automation tools like Ansible can transcend their traditional role of static provisioners, acting instead as continuous security enforcement engines.

7.1 Summary of Contributions

Reflecting upon the initial objectives outlined in Chapter 1, this work delivered several core contributions. First, the research successfully modeled and implemented two competing replication topologies powered by MM2: the Child-Push and the Root-Pull architectures. To secure these deployments, a Macro-Architectural, data-centric Threat Model was formulated using the STRIDE and MITRE ATT&CK frameworks. By defining specific adversarial personas (the Network Adversary, the Malicious Observer, and the Compromised Administrator), the study mapped the vulnerabilities inherent to geographically dispersed, multi-tier environments.

To actively counter these threats, a modular and idempotent Ansible repository was developed to dynamically translate theoretical security policies into technical countermeasures. This effort included the automated generation of an internal Public Key Infrastruc-

ture for mTLS, secret management via Ansible Vault, POSIX file system hardening, and the dynamic injection of Kafka ACLs and Network Quotas.

Crucially, the proposed countermeasures were empirically validated on a live, VPN-routed testbed. The execution of simulated cyber-attacks demonstrated the resilience of the infrastructure: mTLS enforcement successfully dropped unauthorized network probing, POSIX boundaries restricted unprivileged local credential harvesting, and dynamic ACLs combined with Ansible's autonomous drift reconciliation effectively mitigated data poisoning and application-level DoS attacks. Finally, a replication performance benchmark was conducted: the evaluation quantified the physical overhead of the security layers, while the observed computational footprint (CPU and RAM) remains highly sustainable for constrained Edge nodes.

7.2 Architectural Conclusions

The empirical validation process confirmed that there is no single optimal solution for deploying secure hierarchical streaming pipelines; the architectural choice is strictly contextual and depends on the underlying network logistics, resource availability, and trust assumptions.

The Child-Push architecture presents distinct advantages for highly decentralized deployments operating over standard public networks (WANs) characterized by NAT gateways and dynamic IP addresses. It naturally facilitates Edge-to-Cloud connectivity by keeping the peripheral nodes completely hidden from inbound transport traffic. Furthermore, the benchmarking results highlighted its superior horizontal scalability, as the computational burden of encrypted replication is distributed across the Edge. However, this topology demands a highly hardened Cloud core capable of withstanding external network exposure and requires strict, dynamic ACLs to prevent compromised Edge nodes from saturating the central broker.

Conversely, the Root-Pull architecture is highly suitable for tightly integrated enterprise environments (such as dedicated VPN overlays) where the central Cloud infrastructure operates under relaxed Zero Trust assumptions. By centralizing the replication engines on the Core, it completely hides the central broker from the public network, adheres to Kafka's "consume from remote, produce to local" best practice, and provides strong administrative governance over the entire pipeline. However, its 1-to-N orchestration scaling complexity and the necessity to expose peripheral Edge brokers make it unsuitable for unmanaged or highly volatile public transport networks.

Ultimately, by leveraging the proposed Security-as-Code pipeline, both topologies can be securely deployed and autonomously maintained, allowing organizations to adapt their

infrastructure to their specific operational constraints without sacrificing data confidentiality, integrity, or availability.

7.3 Limitations and Future Work

While this thesis established a robust, secure baseline for the Hierarchical ODA ecosystem at the macro-architectural level, certain boundaries were deliberately drawn to focus on infrastructure and transport security. Acknowledging these limitations highlights the natural avenues for future research and engineering enhancements required to elevate the system to a fully mature, enterprise-grade standard:

- **Internal Microservices Security and Input Validation:** The current security model operates under the explicit assumption that the baseline ODA application acts as a trusted "black box". A critical future step is to complete the security implementation directly within the standalone ODA instance. This involves engineering native, fine-grained authentication and authorization mechanisms for Data Generators (DGs) and Data Consumers (DCs) interacting with the API Gateway. Furthermore, strict application-layer input validation must be enforced before any payload is allowed into the local Kafka broker, neutralizing data poisoning attempts right at the source.
- **Comprehensive Logging and Non-Repudiation:** To fully address the Repudiation threat category of the STRIDE framework, the infrastructure requires a more sophisticated auditing mechanism. Future iterations should implement comprehensive, tamper-evident logging across the entire ecosystem. Critical actions, from automated Ansible deployments to individual Data Pump queries and cross-network replications, should be securely logged and cryptographically tied to a specific identity, ensuring full traceability and accountability for all systemic events.
- **Advanced Event-Driven Remediation Logic:** The autonomous drift reconciliation developed in this PoC utilizes Ansible's event-driven handlers. While highly effective for localized file tampering, its current implementation is limited in scope. Future orchestration pipelines should evolve this mechanism to handle more complex scenarios, integrating advanced state-evaluation logic capable of orchestrating multi-step remediation workflows without risking race conditions or inadvertently disrupting healthy tenants during simultaneous scaling events.
- **Dedicated Network Architecture and Hardware Firewalling:** The current network security model relies on software-level VPN overlays and implicit assumptions

regarding the underlying provider's ingress filtering. A necessary future enhancement involves explicit, hardware-level networking design. This requires integrating the deployment pipeline directly with enterprise-grade firewalls and dedicated routing appliances, enforcing strict L3/L4 filtering at the physical perimeter to structurally mitigate volumetric network DoS attacks.

- **Granular Secret Management Expansion:** For operational convenience within the PoC, the infrastructure leveraged a single master password, securely decrypted via Ansible Vault, to unlock multiple keystores, truststores, and private keys. To adhere strictly to the principle of Least Privilege, cryptographic secret management must be expanded. Future deployments should utilize uniquely generated, distinct passphrases for every single cryptographic asset and microservice. Managing this complexity will require expanding the Ansible Vault implementation or integrating dynamic key management systems, further limiting the potential impact of any localized credential exposure.

Appendix A

Appendix

A.1 SSH Configuration

To establish the secure management channel between the trusted Ansible Controller and the untrusted target nodes, a password-less *ED25519* key pair was generated. The specific command executed on the Controller is depicted in Listing A.1.

```
$ ssh-keygen -t ed25519 -C "ansible_controller_wsl" -f ~/.ssh/id_ed25519_ansible
```

Listing A.1: Generation of the passwordless ED25519 key pair on the Ansible Controller.

Once generated, the public key was distributed to the `~/.ssh/authorized_keys` file of the deployment user across the three VMs comprising the infrastructure as shown in Listing A.2.

```
$ ssh-copy-id -i ~/.ssh/id_ed25519_ansible.pub "USER"@IP1,2,3"
```

Listing A.2: Distribution of the public key to the Target Nodes.

As detailed in Listing A.3, to decouple the SSH transport logic from the Ansible playbooks and ensure connection stability over the University's VPN, the OpenSSH client configuration file (`~/.ssh/config`) on the Controller was explicitly hardened. The configuration implements the `ServerAliveInterval` directive to prevent idle timeouts. To mitigate the Adversary-in-the-Middle, the client enforces host key checking. By requiring the servers' public host keys to be pre-validated (e.g., securely gathered out-of-band during the initial VM provisioning) and stored in the Controller's `known_hosts` file, the SSH client proactively drops any intercepted or spoofed connection attempt along the untrusted transport network.

```
1 Host "rootn" "IP1"
2     HostName "IP1"
3     User "USER"
4     IdentityFile ~/.ssh/id_ed25519_ansible
5     ServerAliveInterval 60
6     ServerAliveCountMax 5
7     StrictHostKeyChecking yes
8
9 Host "childn1" "IP2"
10     HostName "IP2"
11     User "USER"
12     IdentityFile ~/.ssh/id_ed25519_ansible
13     ServerAliveInterval 60
14     ServerAliveCountMax 5
15     StrictHostKeyChecking yes
16
17 Host "childn2" "IP3"
18     HostName "IP3"
19     User "USER"
20     IdentityFile ~/.ssh/id_ed25519_ansible
21     ServerAliveInterval 60
22     ServerAliveCountMax 5
23     StrictHostKeyChecking yes
```

Listing A.3: OpenSSH client configuration file used on the Ansible Controller.

A.2 Root-Pull Certificates Generation

In contrast to the Child-Push PKI architecture detailed in Chapter 5, the Root-Pull topology fundamentally inverts the data flow and the associated cryptographic Trust Model. In this configuration, the central Root node acts as the active network client pulling data, meaning it does not need to expose a Kafka broker to the untrusted transport network. Consequently, the MM2 instances running on it require Client Keystores rather than Server Keystores. Conversely, the peripheral Child nodes act as passive servers, exposing their brokers to the Root and thus requiring Server Keystores.

To accommodate this reversed architecture, the automated PKI generation script was adapted with two critical security adjustments:

- **Subject Alternative Name (SAN) Enforcement for Edge Servers:** Because the Child nodes now act as listening servers, their certificates must explicitly include their externally reachable IP addresses within the SAN extension. This is a strict requirement to successfully pass TLS hostname verification during the handshake

process.

- **Granular Client Identities:** To preserve the principle of Least Privilege and enable precise ACLs on the distributed Child brokers, a single, shared client certificate for the Root node is insufficient. Instead, a unique client certificate is generated for each specific MM2 instance residing on the Root. The CN for each client is dynamically derived (e.g., `oda_root_1`) to uniquely and safely identify the central consumer to each specific peripheral Child.

Listing A.4 provides the complete Bash automation script tailored to generate the cryptographic assets for the Root-Pull topology.

```

1 #!/bin/bash
2
3 PASSWORD="XXX"
4 ROOT_CN="oda_root"
5 # Child nodes act as servers, requiring IPs for SAN fields.
6 # Format: "inventory_hostname:IP_Address"
7 CHILD_NODES=("oda_child_1:XXX.XXX.XXX.XXX" "oda_child_2:XXX.XXX.XXX.XXX"
8             ")
9 OUT_DIR="oda_certs_output"
10
11 echo "starting ODA mTLS certificates generation for Root-Pull"
12
13 rm -rf $OUT_DIR
14 mkdir -p $OUT_DIR/ca
15 cd $OUT_DIR
16
17 # 1. CA (Certificate Authority)
18 echo "generating CA"
19 openssl req -new -newkey rsa:4096 -days 3650 -x509 \
20   -subj "/CN=ODA-Internal-CA" \
21   -keyout ca/ca-key -out ca/ca-cert -nodes > /dev/null 2>&1
22
23 # 2. CHILD SERVERS & ROOT MM2 CLIENTS
24 #
25 # The root does not expose the broker, thus it has no server
26 # certificates.
27 # We generate server certs for the children and the corresponding
28 # client certs for the root.
29
30 mkdir -p $ROOT_CN
31
32 for NODE in "${CHILD_NODES[@]}; do
33   CHILD="${NODE%%:*}"
34   CHILD_IP="${NODE##*:}"
35
36   # Extract the index from the child name (e.g., from "oda_child_1" we
37   # get "1")
38   # and use it to create the client CN (e.g., "oda_root_1")
39   CHILD_INDEX="${CHILD##*_}"

```

```

37 CLIENT_CN="oda_root_${CHILD_INDEX}"
38
39 echo "generating server certs for $CHILD and client certs for
    $CLIENT_CN"
40
41 # Create directories respecting the naming convention
42 mkdir -p $CHILD
43 mkdir -p $ROOT_CN/$CHILD
44
45 #
46 # CHILD NODE: SERVER KEYSTORE & TRUSTSTORE
47 #
48 # Truststore
49 keytool -keystore $CHILD/kafka.server.truststore.jks -alias CARoot -
    import -file ca/ca-cert -storepass $PASSWORD -noprompt > /dev/null
    2>&1
50
51 # Keystore and Sign (Child is the server, requires SAN with its IP)
52 keytool -genkey -keystore $CHILD/kafka.server.keystore.jks -alias
    server -dname "CN=$CHILD" \
53     -keyalg RSA -storepass $PASSWORD -keypass $PASSWORD \
54     -ext SAN=DNS:localhost,IP:127.0.0.1,IP:0.0.0.0,IP:$CHILD_IP > /dev/
    null 2>&1
55
56 keytool -keystore $CHILD/kafka.server.keystore.jks -alias server -
    certreq -file $CHILD/server-req -storepass $PASSWORD > /dev/null
    2>&1
57
58 echo "subjectAltName=DNS:localhost,IP:127.0.0.1,IP:0.0.0.0,IP:
    $CHILD_IP" > $CHILD/extfile.cnf
59
60 openssl x509 -req -CA ca/ca-cert -CAkey ca/ca-key -in $CHILD/server-
    req -out $CHILD/server-cert -days 365 -CAcreateserial -passin pass:
    $PASSWORD -extfile $CHILD/extfile.cnf > /dev/null 2>&1
61
62 keytool -keystore $CHILD/kafka.server.keystore.jks -alias CARoot -
    import -file ca/ca-cert -storepass $PASSWORD -noprompt > /dev/null
    2>&1
63 keytool -keystore $CHILD/kafka.server.keystore.jks -alias server -
    import -file $CHILD/server-cert -storepass $PASSWORD -noprompt > /
    dev/null 2>&1
64
65 #
66 # ROOT NODE: MM2 CLIENT KEYSTORE & TRUSTSTORE FOR THIS SPECIFIC CHILD
67 #
68 CLIENT_DIR="$ROOT_CN/$CHILD"
69
70 # Truststore
71 keytool -keystore $CLIENT_DIR/kafka.client.truststore.jks -alias
    CARoot -import -file ca/ca-cert -storepass $PASSWORD -noprompt > /
    dev/null 2>&1
72
73 # Keystore and Sign (Use the new CLIENT_CN)
74 keytool -genkey -keystore $CLIENT_DIR/kafka.client.keystore.jks -
    alias client -dname "CN=$CLIENT_CN" \
75     -keyalg RSA -storepass $PASSWORD -keypass $PASSWORD > /dev/null

```

```
2>&1
76 keytool -keystore $CLIENT_DIR/kafka.client.keystore.jks -alias client
   -certreq -file $CLIENT_DIR/client-req -storepass $PASSWORD > /dev/
   null 2>&1
77 openssl x509 -req -CA ca/ca-cert -CAkey ca/ca-key -in $CLIENT_DIR/
   client-req -out $CLIENT_DIR/client-cert -days 365 -CAcreateserial -
   passin pass:$PASSWORD > /dev/null 2>&1
78 keytool -keystore $CLIENT_DIR/kafka.client.keystore.jks -alias CARoot
   -import -file ca/ca-cert -storepass $PASSWORD -noprompt > /dev/
   null 2>&1
79 keytool -keystore $CLIENT_DIR/kafka.client.keystore.jks -alias client
   -import -file $CLIENT_DIR/client-cert -storepass $PASSWORD -
   noprompt > /dev/null 2>&1
80
81 done
82
83 # Cleaning
84 echo "cleaning csr and temporary files"
85 find . -name "*-req" -delete
86 find . -name "*-cert" -delete
87 find . -name "*.srl" -delete
88 find . -name "*.cnf" -delete
89
90 echo "finished! Output in '$OUT_DIR'"
```

Listing A.4: Automated PKI generation script for the Root-Pull topology.

Bibliography

- [1] W. M. Mohammed, B. R. Ferrer, U. Iftikhar, J. L. M. Lastra, and J. H. Simarro, “Supporting a Cloud Platform with Streams of Factory Shop Floor Data in the Context of the Intustry 4.0,” in *2018 IEEE 16th International Conference on Industrial Informatics (INDIN)*, pp. 786–791, 2018.
- [2] R. Laigner, M. Kalinowski, P. Diniz, L. Barros, C. Cassino, M. Lemos, D. Arruda, S. Lifschitz, and Y. Zhou, “From a Monolithic Big Data System to a Microservices Event-Driven Architecture,” in *2020 46th Euromicro Conference on Software Engineering and Advanced Applications (SEAA)*, pp. 213–220, 2020.
- [3] F. Bosi, A. Corradi, G. Di Modica, L. Foschini, R. Montanari, L. Patera, and M. Solimando, “Enabling Smart Manufacturing by Empowering Data Integration with Industrial IoT Support,” in *2020 International Conference on Technology and Entrepreneurship (ICTE)*, pp. 1–8, 2020.
- [4] H. Pan, R. Chard, S. Zhou, A. Kamatar, R. Vescovi, V. Hayot-Sasson, A. Bauer, M. Gonthier, K. Chard, and I. Foster, “Octopus: Experiences with a Hybrid Event-Driven Architecture for Distributed Scientific Computing,” in *SC24-W: Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pp. 496–507, 2024.
- [5] C. Pahl, N. G. Gunduz, O. C. Sezen, A. Ghamgosar, F. Hofer, and N. El Ioini, “A Systematic Review of Infrastructure-as-Code Technologies,” in *Communications in Computer and Information Science (CCIS)*, Springer, 2025.
- [6] I. Aviv, R. Gafni, S. Sherman, B. Aviv, A. Sterkin, and E. Bega, “Infrastructure From Code: The Next Generation of Cloud Lifecycle Automation,” *IEEE Software*, vol. 40, no. 1, pp. 42–49, 2023.
- [7] Red Hat, Inc., “Ansible Documentation.” <https://docs.ansible.com/>, 2026. Accessed: 5 March 2026.

- [8] A. Rahman, F. L. Barsha, and P. Morrison, “Shhh!: 12 Practices for Secret Management in Infrastructure as Code,” in *2021 IEEE Secure Development Conference (SecDev)*, pp. 56–62, 2021.
- [9] W. Hummer, F. Rosenberg, F. Oliveira, and T. Eilam, “Testing Idempotence for Infrastructure as Code,” in *Proceedings of the 14th ACM/IFIP/USENIX International Conference on Middleware*, pp. 368–388, Springer, 2013.
- [10] R. Opdebeeck, A. Zerouali, and C. De Roover, “Behaviour-Aware Security Smell Detection for Infrastructure as Code,” in *Proceedings of the 22nd Belgium-Netherlands Software Evolution Workshop (BENEVOL)*, CEUR Workshop Proceedings, 2023.
- [11] D. Dodson, M. Souppaya, and K. Scarfone, “Secure Software Development Framework (SSDF) Version 1.1: Recommendations for Mitigating the Risk of Software Vulnerabilities,” Tech. Rep. NIST Special Publication (SP) 800-218, National Institute of Standards and Technology (NIST), 2022.
- [12] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, “Zero Trust Architecture,” NIST Special Publication (SP) 800-207, National Institute of Standards and Technology (NIST), August 2020.
- [13] A. Bocci and S. Forti, “ODA - Observable Data Access.” <https://github.com/di-unipi-socc/ODA>, 2025. Accessed: 23 February 2026.
- [14] A. Bocci, “ODA - Observable Data Access, Version 1.1.” https://github.com/alebocci/ODA_1.1, 2025. Accessed: 23 February 2026.
- [15] Apache Software Foundation, “Apache Kafka 3.6.X Documentation.” <https://kafka.apache.org/36/getting-started/introduction/>, 2025. Accessed: 4 March 2026.
- [16] J. Kreps, N. Narkhede, J. Rao, *et al.*, “Kafka: A distributed messaging system for log processing,” in *Proceedings of the NetDB*, vol. 11, pp. 1–7, 2011.
- [17] Apache Software Foundation, “Geo-Replication (Cross-Cluster Data Mirroring).” <https://kafka.apache.org/36/operations/geo-replication-cross-cluster-data-mirroring>, 2025. Accessed: 4 March 2026.
- [18] L. Kohnfelder and P. Garg, “The threats to our products.” Microsoft Internal Document, 1999. Microsoft Corporation.
- [19] The MITRE Corporation, “MITRE ATT&CK Framework.” <https://attack.mitre.org/>, 2026. Accessed: 22 March 2026.

- [20] Docker Inc., “Docker Compose Documentation.” <https://docs.docker.com/compose/>, 2026. Accessed: 21 March 2026.
- [21] N. Venturi, “Analyzing Microservices: Threat Modeling and Exploitation in an ODA Case Study,” Master’s thesis, University of Pisa, 2025. ETD Archive: etd-11032025-160354.
- [22] Apache Software Foundation, “KIP-382: MirrorMaker 2.0.” <https://cwiki.apache.org/confluence/display/KAFKA/KIP-382%3A+MirrorMaker+2.0>, 2023. Accessed: 4 March 2026.
- [23] OWASP Foundation, “Threat Modeling Process.” https://owasp.org/www-community/Threat_Modeling_Process, 2026. Accessed: 24 February 2026.
- [24] OWASP Foundation, “Threat Modeling Cheat Sheet.” https://cheatsheetseries.owasp.org/cheatsheets/Threat_Modeling_Cheat_Sheet.html, 2026. Accessed: 24 February 2026.
- [25] W. dos Reis Bezerra, C. A. de Souza, C. M. Westphall, and C. B. Westphall, “HyVAW: A hybrid vulnerability analysis workflow threat model methodology for complex systems based on distributed components,” *Journal of Open Innovation: Technology, Market, and Complexity*, vol. 11, no. 4, p. 100654, 2025.
- [26] S. Nagasundari, P. Manja, P. Mathur, and P. B. Honnavalli, “Extensive Review of Threat Models for DevSecOps,” *IEEE Access*, vol. 13, pp. 45252–45271, 2025.
- [27] M. Souppaya and M. Scarfone, “Guide to Data-Centric System Threat Modeling,” Initial Public Draft NIST Special Publication (SP) 800-154 (Draft), National Institute of Standards and Technology (NIST), 2016. Accessed: 13 March 2026.
- [28] Threat Modeling Manifesto Working Group, “The Threat Modeling Manifesto.” <https://www.threatmodelingmanifesto.org/>, 2020. Accessed: 12 March 2026.
- [29] A. Shostack, “The Ultimate Beginner’s Guide to Threat Modeling.” <https://shostack.org/resources/threat-modeling>, 2026. Accessed: 13 March 2026.
- [30] N. Mead, F. Shull, K. Vemuru, and O. Villadsen, “A Hybrid Threat Modeling Method,” Tech. Rep. CMU/SEI-2018-TN-002, Carnegie Mellon University, Software Engineering Institute, March 2018. Accessed: 13 March 2026.

-
- [31] Joint Task Force Transformation Initiative, “Guide for Conducting Risk Assessments,” Tech. Rep. NIST Special Publication (SP) 800-30 Rev. 1, National Institute of Standards and Technology (NIST), September 2012. Accessed: 13 March 2026.